

UNIVERSIDADE DE LISBOA
FACULDADE DE CIÊNCIAS
DEPARTAMENTO DE INFORMÁTICA



Semantic Indexing of Descriptors, Partitioning and Classification for Mapping Biomedical Entities

João Pedro Ramos Vedor

Mestrado em Engenharia Informática

Dissertação orientada por:
Professor Doutor Francisco José Moreira Couto

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor Francisco Couto, for his guidance, patience, and trust throughout this work. I am deeply thankful to my colleague Sofia for the countless, careful readings of this thesis.

I am indebted to my family for their unwavering patience and support: to my mother, for the many times she gave up the car so I could get home faster; to my grandmother, for the innumerable lunches she prepared; to my father, for his conversations and encouragement; and to my brother, for the humour that lightened long days. I am also grateful to my closest friends—Gonçalo, Mariana, João L., and João R.—for listening with such patience as I talked about this project.

Finally, I dedicate this thesis to my dog, Pink, who passed away during its making.

Abstract

Imagine reading a clinical note and encountering the term *ALS*.” Is it *amyotrophic lateral sclerosis* (a disease), *advanced life support* (a procedure), or even a gene symbol? Traditional **Named Entity Recognition (NER)** can identify that *ALS*” is a mention of a biomedical entity, but it does not disambiguate which specific concept it refers to. **Named Entity Linking (NEL)** is required to map the mention to a unique identifier in an ontology such as UMLS or MeSH, ensuring that downstream systems understand precisely which disease, procedure, or gene is referenced. For example, correctly linking “ALS” to the UMLS concept for amyotrophic lateral sclerosis allows consistent retrieval, reasoning, and integration across biomedical datasets. Scaling NEL across articles, clinical narratives, and curated resources is challenging due to the size of ontologies, acronym collisions, synonyms, and ambiguous mentions (e.g., “Lou Gehrig’s disease” vs. “ALS”). Motivated by these challenges, this thesis introduces **XMR4EL**, a modular and reproducible framework that treats NEL as **eXtreme Multi-label Ranking (XMR)**: “organize → route → rank.” XMR4EL decouples semantic indexing, hierarchical routing, and label-level ranking behind stable interfaces, enabling plug-and-play components while preserving deterministic preprocessing, sparse-first modeling, and persisted artifacts for reproducibility. On automatically labeled disease corpora (Inst-100/Inst-500 for training; BC5CDR for testing), a 4-layer hierarchy achieves 61.7% Hit@100 at 195 ms/mention with beam=40, revealing a clear trade-off point between speed and quality near beams 30–40. Increasing per-label synonyms from 100 to 500 instances yields +8–13 Hit@100 points at practical beams, and added depth improves recall at matched latency by shrinking leaf scopes. **Conclusion:** XMR4EL demonstrates that an open, sparse-first XMR design is practical today. By combining effective mention detection (NER) with accurate concept grounding (NEL), it provides a reliable pipeline for linking disease mentions to their unique IDs, supporting high-quality biomedical information retrieval and integration. Opportunities for further improvements include calibration, document-level coherence, and label-side text encoders to boost top-1/5 accuracy.

Keywords: Biomedical Entity Linking; Extreme Multi-label Ranking; Candidate Generation; Beam Search; Hard Negative Mining

Resumo

Imagina que lê “amyloidosis” num registo clínico. Um sistema de *Named Entity Recognition* (NER) reconhece a palavra e diz “é uma doença”; um sistema de *Named Entity Linking* (NEL) tem de dar o próximo passo: apontar exatamente qual doença no universo de UMLS/MeSH, onde há centenas de milhares de rótulos, múltiplos sinónimos e homónimos, variação morfológica e acrónimos ambíguos (“AL”, “AA”, “ATTR”).

Neste trabalho a normalização de entidades é encarada como um problema de *eXtreme Multi-label Ranking* (XMR), em que o objetivo é produzir, de forma escalável, um conjunto curto e certo de candidatos onde o rótulo correto esteja quase sempre presente, respeitando limites explícitos de latência e memória que viabilizam integração *downstream* em motores de pesquisa, analítica e sistemas de apoio à decisão.

Começo pelo “porquê”. A escala e o enviesamento de cauda longa das ontologias biomédicas tornam inviável pontuar todos os rótulos para cada menção: a distribuição é fortemente desequilibrada, muitos conceitos são raros e novos sinónimos surgem continuamente, o que torna dispendioso (e pouco robusto) um varrimento exaustivo do espaço de rótulos a cada consulta.

Ao mesmo tempo, implementações abertas, modulares, de fácil inserção e substituição de diferentes modelos são raras — o que dificulta comparações justas, análises de **ablação** e reprodutibilidade. **Para responder a estes desafios**, nasce o **XMR4EL**: uma ferramenta que organiza a tarefa em quatro blocos independentes e substituíveis, com atenção à memória, à determinação por semente e à transparência dos modelos e dados, preservando mapas de IDs, partições, protótipos, índices para que cada execução seja repetível e auditável.

O “como” é descrito como um percurso em quatro etapas. Primeiro, procede-se à indexação semântica das menções: o texto é limpo e normalizado, harmonizam-se casos e sinais, detetam-se e separam-se abreviações quando aplicável e geram-se vetores híbridos que combinam *Term Frequency–Inverse Document Frequency* (TF–IDF) (palavras e n -grams de caracteres) com representações densas de transformadores biomédicos adequadas ao domínio; e, crucialmente, o bloco denso é exportado para forma esparsa (quantização/projeção), mantendo operações rápidas e parcimoniosas em memória sem sacrificar capacidade discriminativa nos vizinhos mais próximos.

Em seguida, constroem-se protótipos de rótulos com *Positive Instance Feature Aggregation* (PIFA) — soma de menções positivas seguida de normalização — que passam a viver no mesmo espaço das menções e funcionam como âncoras semânticas para cada conceito, capturando regularidades lexicais e contextuais sem depender de exemplos negativos exógenos.

Depois, procede-se ao particionamento do espaço de rótulos com *spherical k-means* balanceado e hierárquico. Caso o particionamento se revele instável (com aglomerados demasiado pequenos ou desequilibrados), a camada é abortada e o utilizador é alertado para reduzir o número de grupos; privilegia-se a falha precoce em vez da propagação de erros para camadas profundas. Cada rótulo permanece numa única partição por camada, o que evita duplicação de pontuações e torna previsível o custo de inferência por nível.

Avança-se para o combinador: um classificador um-contra-todos leve aprende $p(\text{cluster} \mid x)$ e, com pesquisa por feixe, encaminha cada menção apenas para algumas ramificações promissoras. Este constitui o *controlo de escala*: em vez de avaliar $O(L)$ candidatos por menção, realizam-se $O(\log L)$ decisões de encaminhamento e avaliam-se localmente apenas os casos relevantes, reduzindo a competição a vizinhos plausíveis; na prática, B (largura do feixe) e D (profundidade) tornam-se parâmetros explícitos de custo/qualidade, permitindo traçar *curvas de operação* sob orçamentos de milissegundos. **Este desenho torna explícitos os parâmetros de custo e viabiliza curvas qualidade-latência alinhadas com SLAs restritos.**

Para assegurar estabilidade entre camadas, cada nó emite pequenos resumos de pontuações (média/soma/máximo), que são anexados às características de menções e rótulos nos filhos, fornecendo contexto sem aumentar excessivamente a dimensão e atenuando o efeito cascata de decisões ambíguas.

No final do percurso situam-se os classificadores por rótulo. Aqui, a qualidade dos negativos distingue um bom classificador de um medíocre. Por isso, adota-se um currículo incremental *Teacher-Forcing Negatives* (TFN) $\rightarrow$ *Matcher-Aware Negatives* (MAN): inicia-se com uma mistura de negativos fáceis (aleatórios no aglomerado) e “quase-iguais” (próximos do protótipo por cosseno) e evolui-se para MAN, onde são extraídos negativos difíceis dentro do mesmo aglomerado com busca vetorial (MIPS/ANN) guiada pelo próprio *matcher*, expondo o modelo a confusões reais e ampliando as margens locais. Como o conjunto de negativos evolui a cada ciclo de treino, os modelos são treinados com *partial_fit*, sem re-inicializações, e o protótipo do rótulo é adicionado às características $[x; z_\ell]$ para aprender interações menção $\leftrightarrow$ rótulo, melhorando a discriminação perante sinónimos fortes e diferenças ortográfica.

Na implementação, fixam-se sementes, limitam-se *threads* de BLAS e arquivam-se versões de dados/artefactos, garantindo que os resultados podem ser reproduzidos *bit-a-bit* em máquinas distintas. Quanto aos resultados, mesmo avaliando apenas a geração de candidatos (sem codificador cruzado nem reclassificador final), a hierarquia revela-se eficaz.

Num cenário de doenças com universo memória-limitado de $\sim 6\,000$ **rótulos**, o modelo de quatro camadas alcança **61,7% Hit@100** com cerca de **195 ms/menção** a *beam* 40, exibindo um ponto de compromisso claro de compromisso qualidade-latência entre *beam* 30–40; a partir desse ponto, os ganhos de cobertura por alargamento do feixe tornam-se marginais face ao custo adicional. O aumento do número de exemplos por rótulo de Inst-100 para Inst-500 acrescenta **+8–13** pontos de Hit@100 em feixes práticos, sobretudo quando as menções são ambíguas ou raras; a maior profundidade ajuda por encolher folhas (menos rótulos a pontuar localmente), enquanto o feixe permanece o principal parâmetro de custo e de contenção de erros de corte precoce. Nas medições de tempo, a codificação textual domina a latência total, ao passo que a avaliação local nas folhas e a fusão dos candidatos apresentam custo quase linear no

tamanho da folha, justificando o desenho “esparso-primeiro”.

Demonstra-se esta ferramenta com integração direta em *Hybrid X-Linker*: o *Prediction for Enormous and Correlated Output Spaces* (PECOS) é substituído por módulos XMR4EL, mantendo-se as mesmas interfaces para abreviação, *string matching* e *Personalized PageRank* (PPR), sem alteração da lógica, **o que comprova a modularidade e facilita ablações reprodutíveis**.

Em seleção global, o PECOS conserva vantagem como rastreador puro em *top-1/5*, ao passo que o XMR4EL apresenta *recall@k* razoável com custo controlável e oferece *knobs* de engenharia claros (profundidade/feixe/modo de seleção) e transparência dos modelos e dados para auditoria.

A base modular e reprodutível constitui um objetivo em si: cada decisão (de sementes a mapas de IDs e partições) permanece persistida, e cada bloco (codificação, partição, combinador, classificador) pode ser trocado sem reescrever os restantes, viabilizando comparações justas entre variantes (p. ex., outros codificadores biomédicos, partições com diferentes *balance constraints* ou *merging* de folhas) e estudos de sensibilidade (p. ex., impacto de *k* na fusão *per_leaf* vs. *global*).

Assinalam-se ainda limitações e caminhos de melhoria: a ordenação final é, propositadamente, frugal — coseno vs. protótipos — para evidenciar o poder do gerador; tal opção deixa margem para um reclassificador leve focado nos candidatos já recolhidos, com custo amortizado e ganhos esperados em *top-1/5*; a calibração (p. ex., ECE/NLL) pode alinhar probabilidades entre níveis e melhorar margens sob diferentes SLAs; a coerência documental antes do PPR pode reduzir micro-erros em menções repetidas; e a expansão para universos acima de 6k rótulos permanece compatível com o mesmo desenho, desde que se mantenha a exportação denso→esparsa e folhas pequenas.

Em termos práticos, fica traçado um caminho curto para métricas finais (*Acc@1/5*, *nDCG*) — o vetor global de pontuações encontra-se disponível após a fusão — e assegura-se encaixe em ambientes de produção, onde a previsibilidade do custo por consulta é tão importante quanto a cobertura.

Em síntese, demonstra-se que um desenho esparso-primeiro, com partições navegáveis e feixe controlado, é suficiente para operar à escala de UMLS/MeSH com latências de centenas de milissegundos por menção, devolvendo alavancas explícitas para gerir compensações qualidade–latência e promovendo comparabilidade com modelos partilháveis e postos de controlo verificáveis; os números atuais (p. ex., **61,7% Hit@100** com **~195 ms/menção** a *beam* 40) confirmam a viabilidade e apontam para melhorias adicionais com reclassificadores, calibração e coerência documental, sem sacrificar memória, tempo nem reprodutibilidade.

Palavras-chave: Ligação de Entidades Biomédicas; Classificação Extrema Multi-rótulo; Geração de Candidatos; Pesquisa por Feixe; Amostragem de Negativos Difíceis

Contents

List of Abbreviations	XV
1 Introduction	1
1.1 Motivation	1
1.2 Problem	2
1.3 Objectives	3
1.4 Methodology	4
1.4.1 Experimental Setup: Reproducibility and Hardware	6
1.5 Contributions	7
1.6 Document Structure	8
2 Related Work	9
2.1 Biomedical Named Entity Linking (NEL): Overview and Fundamentals	9
2.2 Extreme Multi Label Learning	11
2.3 Representations for Mentions and Labels	12
2.4 Partitioning and Routing: Organizing a Million Labels into a Few Good Options	14
2.5 Rankers and Negative Mining at Scale	15
2.6 Evaluation and Calibration (NEL, XMC, and XMR)	16
2.7 Hybrid X-linker	17
3 eXtreme Multi Label Ranking for Entity linking (XMR4EL)	21
3.1 Representations and supervision	21
3.2 Label-space organisation	23
3.3 Matcher (classifier-based routing)	24
3.4 Label-wise ranking	24
3.5 Hierarchical training	27
3.6 Inference	28
3.7 Efficiency, memory, and systems considerations	30
3.8 Hybrid X-Linker: Scope of modifications	31
3.9 Results and Discussion	32
4 Conclusion	37

References	39
A Model Configurations and Experimental Setup	43

List of Figures

1.1	Model Organise→route→rank pipeline	5
2.1	Minimal PECOS-style pipeline	12
2.2	Hybrid X-Linker (inference)	17
3.1	Label-space organisation	24
3.2	Classifier-based routing over K clusters	25
3.3	Label-wise ranking within a cluster	26
3.4	Multi-layer training across depths	28
3.5	Inference with budgeted expansion	29
3.6	Hybrid X-Linker (inference) — XMR4EL	31
3.7	Beam width vs quality and latency (D=4, Top- k =100)	33

List of Tables

1.1	XMR4EL pipeline stages	6
2.1	Core biomedical KBs/ontologies and KOS	10
2.2	Biomedical EL corpora	11
2.3	Trade-offs across extreme families	12
2.4	Evaluation map for NEL/XMC/XMR	16
2.5	Hybrid X-Linker training files	18
3.1	Representation artefacts and formats	23
3.2	Matcher configuration	25
3.3	Ranker configuration	26
3.4	Depth vs latency at $B=10$	32
3.5	Coverage vs list length ($D=4$, $B=40$)	33
3.6	Impact of per-label supervision (diagnostic split)	34
3.7	Latency breakdown by stage ($D=3$, $ \mathcal{Y} =1000$)	34
3.8	Latency breakdown by stage ($D=4$, $ \mathcal{Y} =1000$)	35
3.9	Hybrid X-Linker: PECOS vs XMR4EL (global selection)	35

List of Abbreviations

ACC@1/5	Accuracy at top-1 / top-5 XIV
ANN	Approximate Nearest Neighbour XIV , 7 , 11 , 15 , 30
API	Application Programming Interface XIV , 31
BERT	Bidirectional Encoder Representations from Transformers XIV
BioBERT	Biomedical BERT XIV , 13 , 23
BM25	Best Match 25 XIV , 10 , 13
CPU	Central Processing Unit XIV , 11 , 30
CSC	Compressed Sparse Column XIV , 7 , 22 , 30
CSR	Compressed Sparse Row XIV , 7 , 22 , 30
CTD	Comparative Toxicogenomics Database XIV , 5 , 10 , 17
CUI	Concept Unique Identifier XIV , 10
ECE	Expected Calibration Error XIV , 17
EL	Entity Linking XIII , XIV , 2 , 3 , 7 , 8 , 9 , 10 , 11 , 15 , 17 , 18 , 19
GMEV	General Matrix Vector Multiply XIV , 30
GPU	Graphics Processing Unit XIV , 30
HNSW	Hierarchical Navigable Small World XIV , 11 , 15
ID	Identifier XIV
KB	Knowledge Base XIII , XIV , 9 , 10 , 17 , 18 , 31 , 32
kNN	k -Nearest Neighbours XIV
KOS	Knowledge Organization System XIII , XIV , 10 , 18 , 21 , 22
MAN	Matcher-Aware Negatives XIV , 7 , 25

MAP Mean Average Precision [XIV, 16](#)

MEDIC CTD MEDIC disease vocabulary [XIV, 5, 10, 17](#)

MeSH Medical Subject Headings [XIV, 1, 2, 6, 9, 10, 15, 18](#)

MIPS Maximum Inner Product Search [XIV, 7, 30](#)

MLP Multilayer Perceptron [XIV, 30](#)

MRR Mean Reciprocal Rank [XIV, 16](#)

nDCG@ k Normalized Discounted Cumulative Gain at cut-off k [XIV, 2, 3, 5, 11, 16](#)

NEL Named Entity Linking [III, XIII, XIV, 1, 2, 9, 10, 12, 13, 16](#)

NER Named Entity Recognition [III, XIV, 9](#)

NFC Normalization Form Canonical Composition [XIV, 21](#)

NFKC Normalization Form Compatibility Composition [XIV, 21](#)

NLL Negative Log-Likelihood [XIV, 17](#)

NLM National Library of Medicine [XIV, 10](#)

OMIM Online Mendelian Inheritance in Man [XIV, 10](#)

OvR One-vs-Rest [XIV, 7, 8, 9, 11, 12, 24, 27](#)

P@ k Precision at cut-off k [XIV, 16, 17](#)

PECOS Prediction for Enormous and Correlated Output Spaces [XI, XIII, XIV, 4, 7, 8, 9, 11, 12, 13, 17, 18, 25, 31, 35, 37, 38](#)

PIFA Positive Instance Feature Aggregation [XIV, 4, 7, 13, 22, 37](#)

PLT Probabilistic Label Tree [XIV, 9, 11, 12, 14](#)

PPR Personalized PageRank [XIV, 17, 31, 35, 38](#)

RAM Random Access Memory [XIV](#)

SapBERT Synonym-aware pre-training of BERT [XIV, 13, 14, 17, 19](#)

SVD Singular Value Decomposition [XIV, 14, 22, 23, 30](#)

SVM Support Vector Machine [XIV, 24](#)

TF-IDF Term Frequency-Inverse Document Frequency [XIV, 4, 12, 13, 23, 30](#)

TFN Teacher-Forcing Negatives [XIV, 7, 25](#)

UMLS Unified Medical Language System [XIV](#), [1](#), [2](#), [9](#), [10](#), [13](#)

XMC eXtreme Multi-label Classification [XIII](#), [XIV](#), [1](#), [2](#), [8](#), [9](#), [11](#), [16](#)

XMR eXtreme Multi-label Ranking [III](#), [XIII](#), [XIV](#), [2](#), [3](#), [4](#), [7](#), [8](#), [9](#), [11](#), [12](#), [16](#), [31](#)

XMR4EL XMR for Entity Linking [XI](#), [XIII](#), [XIV](#), [6](#), [21](#), [31](#), [32](#), [33](#), [35](#), [36](#), [37](#), [38](#)

Chapter 1

Introduction

This chapter motivates the problem, states my objectives, outlines the methodology at a high level, summarizes the main contributions, and describes the document structure.

1.1 Motivation

A scientist reads a paper and encounters the term “*amyloidosis*.” Determining whether this refers to “AL amyloidosis,” “AA amyloidosis,” or another subtype—and linking it to a standard identifier in MeSH or UMLS—is critical, because misassignments propagate errors in literature mining, clinical decision support, and automated knowledge graphs.

The session begins routinely: query PubMed (a biomedical literature database), skim abstracts, note variants from titles and headings (“AL,” “AA,” “ATTR”), open the MeSH Browser (Medical Subject Headings, a controlled vocabulary) to inspect tree positions and entry terms, consult the UMLS Metathesaurus (Unified Medical Language System, integrating multiple vocabularies with unique concept identifiers, CUIs) to align identifiers across sources, and, when the meaning is subtle, turn to reviews to decide whether a phrase denotes the systemic disease, a specific subtype, or organ-limited involvement. Only then is it safe to assign the canonical identifier—one MeSH descriptor (and, if needed, a UMLS CUI) that other tools will interpret consistently.

This meticulous workflow accomplishes *grounding*: surface forms become ontology-backed, reusable concepts that allow downstream tasks—literature curation, clinical decision support, knowledge-base construction—to operate on knowledge rather than strings. Yet doing this by hand is infeasible at modern scale; the literature and curated resources expand faster than human curation can keep pace.

To scale the scientist’s workflow, the community formalised the task as Named Entity Linking (NEL): given a textual mention, return its canonical ontology identifier (e.g., a MeSH descriptor or UMLS CUI) so that interoperability is preserved across systems and datasets [Shen et al. \[2014\]](#).

In parallel, learning with very large output spaces matured into eXtreme Multi-label Classification (XMC): algorithms designed to predict over label sets in the 10^5 – 10^6 range by combining indexing, hier-

archical or graph-based decompositions, and retrieval-aware objectives Prabhu et al. [2018]; Khandagale et al. [2020]; Jiang et al. [2021]. Within XMC, a complementary formulation is eXtreme Multi-label Ranking (XMR), which focuses on *retrieving and ordering* a small candidate set from the extreme label universe.

XMC emphasises ranking metrics such as $\text{Hit}@k$ and $\text{nDCG}@k$, cleanly separates *candidate generation* from *final assignment*, and enables systems to score only compact slices of the label space while maintaining coverage of rare labels.

Putting this in perspective with the scientist’s routine, XMR operationalises the same three moves that occur during manual curation. First, the concept space is **organised**—much like browsing MeSH trees or UMLS relations to form a mental index of nearby ideas. Second, a mention is **routed** to a plausible neighbourhood—analogue to narrowing attention to a subtree such as “amyloidosis” and its subtypes (AL, AA, ATTR). Third, near neighbours are **ranked** with fine granularity—akin to deciding whether the text denotes the systemic disease, a subtype, or an organ-limited variant based on definitions and usage patterns. Framed this way, NEL inherits mature large-output tools while preserving the core EL contract of returning a single canonical identifier and supporting reproducible evaluation on biomedical corpora such as MedMentions and BC5CDR Mohan and Li [2019]; Peng et al. [2019].

1.2 Problem

Translating this framing into a deployed, ontology-scale system exposes several coupled constraints. The label set is massive—hundreds of thousands of MeSH/UMLS concepts—so exhaustive per-mention scoring is infeasible. To make search tractable, the label space is indexed, and each mention is *routed* through a hierarchical structure, pruning most candidates.

However, pruning is not disambiguation: even inside a routed cell (a cluster of plausible candidate labels), neighbourhoods remain dense with near-duplicates, fine-grained subtypes, and organ-limited variants that all plausibly match the same string. Abbreviations and synonymy collapse many surface forms into overlapping regions, and corpus conventions drift over time. This makes literal matching or narrow context brittle Shen et al. [2014]; Peng et al. [2019].

In short, routing is necessary for scale but insufficient to decide which concept is correct—misassignments could propagate errors in downstream tasks such as literature mining, knowledge-base curation, or clinical decision support. Routing is like narrowing a search to a subtree of closely related concepts; fine-grained ranking is still required to select the exact label.

Learning dynamics amplify these pressures. The frequency distribution of labels is long-tailed: a small head dominates training signal, while most labels have few or no explicit examples. Without care, representation learning drifts toward the head, and ranking inside routed neighbourhoods becomes brittle for the tail.

Prototype construction, negative sampling, and curriculum design must therefore supply informative contrast where annotated signal is scarce to maintain accuracy across the entire ontology.

Even if the model could learn perfectly, real-time deployment introduces additional operational constraints. Inference must meet strict latency and memory budgets typical of commodity hardware; candidate generation and routing require explicit “speed–quality” controls whose behaviour is predictable under workload; and depth in the hierarchy improves recall only if routing overhead does not dominate end-to-end time. These systems also live in ecosystems that demand reproducibility: preprocessing should be deterministic; partitions, mappings, and model artifacts should be persisted; and evaluation should report retrieval metrics such as $\text{Hit}@k$ and $\text{nDCG}@k$, which serve as clear upper bounds for any downstream assignment—ensuring results are comparable across components, datasets, and over time [Shen et al. \[2014\]](#); [Jiang et al. \[2021\]](#).

Finally, to make progress cumulative rather than bespoke, implementations must be modular and plug-and-play: swapping encoders, routers, or rankers should not alter data handling, artifact formats, or metric definitions, and should preserve the entity-linking contract of returning a single, canonical identifier [Kolitsas et al. \[2018\]](#).

Taken together, these challenges highlight the need for methods that balance accuracy, efficiency, and reproducibility across a massive and ambiguous label space. A system must not only prune and rank effectively but also provide modularity and deterministic behavior to support consistent evaluation and iterative improvement.

1.3 Objectives

Given the scale and disambiguation pressures outlined above, the goal is to make ranking-first methods practical and comparable for biomedical entity spaces. I treat entity normalization as extreme-label *ranking* over ontology identifiers, focusing on non-contextual, mention-level inputs (e.g., synonym strings), where scalability, label imbalance, and fine-grained discrimination are the primary challenges.

Concretely, the thesis pursues four objectives:

1. **Design a modular, model-agnostic, XMR pipeline for EL-scale ontologies.** Separate concerns into (i) representation, (ii) partitioning and hierarchical routing (beam search), and (iii) leaf-level scoring with configurable fusion of sparse/dense signals—so that components can be swapped without restructuring the system, preserving reproducibility and comparability.
2. **Deliver efficiency and reproducibility at scale.** Implement memory-aware indexing and traversal, favor deterministic preprocessing and seeding, drive experiments via configuration files, and provide a standardized evaluation harness oriented to retrieval metrics ($\text{Hit}@k$, $\text{nDCG}@k$), with head/tail breakdowns to assess performance across common and rare concepts.
3. **Enable systematic ablations and fair comparison.** Expose stable interfaces for alternative encoders, clustering/partitioning schemes, lexical matchers, fusion rules, and negative sampling policies, allowing apples-to-apples studies under a consistent training and inference pipeline.

4. **Open-source the stack by removing external dependencies.** Make *Hybrid X-Linker* Ruas et al. [2024] fully open source by replacing PECOS Yu et al. [2022] with this thesis XMR components, while preserving the original control flow (abbreviation expansion $\rightarrow$ lexical matching $\rightarrow$ hierarchical routing $\rightarrow$ leaf scoring/fusion).

Scope-wise, this thesis emphasizes candidate generation over ontology identifiers and evaluates retrieval quality as an upper bound for any downstream scorer (e.g., on MedMentions Mohan and Li [2019] and BC5CDR Li et al. [2016] datasets). Extensions to contextual EL and calibration for final assignment are deferred to future work.

1.4 Methodology

Scope: The system treats mention-level normalisation as retrieval and ranking over a biomedical ontology. A surface form, with no document context, is taken as a query; the goal is to return a compact, ordered set of identifiers that downstream components can consume. The design follows the curator’s routine in computational form: first the concept space is organised so that related labels live near one another; then a router directs each mention into a small neighbourhood where confusion is plausible; finally a lightweight ranker decides among these near neighbours with fine granularity. Emphasis is placed on modularity, determinism, and a sparse-first approach so results are repeatable on commodity hardware. A compact view of this workflow is shown in Figure 1.1, and the stages/artefacts are summarised in Table 1.1.

Mentions and labels share a common space to make nearest-neighbour reasoning cheap and stable (see the *Representations* row in Table 1.1). Each mention x is mapped by a lightweight encoder, which concatenates a sparse TF-IDF block with a transformer embedding and then ℓ_2 normalises so cosine reduces to a dot product. Labels inhabit the same space via Positive Instance Feature Aggregation (PIFA) giving a unit-norm prototype per concept.

Organisation proceeds in stages from the prototype matrix Z (cf. *Organisation* in Table 1.1 and Figure 1.1). Labels are clustered under cosine geometry to form compact regions in which genuine confusions occur. A balanced spherical k -means variant keeps child sizes predictable; when balance cannot be guaranteed, seeded spherical k -means yields deterministic assignments. Splits are single-membership by design—each label belongs to exactly one child at a depth—simplifying routing and making the hierarchy easy to persist. Learned partitions and ID maps are treated as first-class artefacts and reused across training and evaluation.

Routing answers “*where to look first?*” (see the *Routing* row in Table 1.1 and the middle of Figure 1.1). At each non-leaf node a multiclass, one-vs-rest linear matcher $M^{(h)}$ is trained on node-relevant mentions and returns non-negative child scores $\pi_c^{(h)}(x)$. Scores are not forced to sum to one; only their relative order matters for beam expansion. Because most children are negatives for most mentions, class

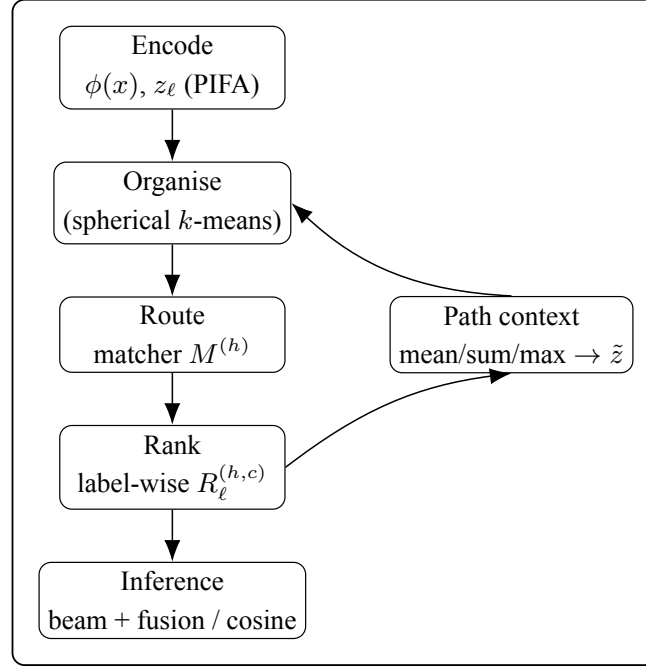


Figure 1.1: Compact pipeline with *organise* $\rightarrow$ *route* $\rightarrow$ *rank*, path-aware feedback to improve deeper splits, and beam-based inference for efficient candidate generation.

imbalance is handled in the learner. Inside each child, the focus shifts to “*who wins?*”: small, label-wise rankers $\{R_\ell^{(h,c)}\}$ operate on local positives with in-cluster negatives, learning margins that reflect real confusions within the neighbourhood (cf. *Local ranking* in Table 1.1).

Evidence does not stop at node boundaries. To let deeper decisions benefit from earlier ones without inflating dimensionality, a compact path summary is passed forwards (the *Path context* box in Figure 1.1). For a label ℓ routed through child c at layer h , routing confidence and (when present) local ranking are fused per mention and then distilled over *training* positives. These are z-scored within the node and concatenated to the base prototype before re-normalisation, yielding a path-aware prototype $\tilde{z}_\ell^{(h)}$ that the next split will see. In practice this stabilises deeper clustering and focuses rankers on distinctions that actually arise along routed paths.

Inference mirrors training with a strict budget (right-hand side of Figure 1.1; see also the *Inference* row in Table 1.1). The query is encoded once and pushed through the hierarchy with a beam: at each layer only the top b_h children by accumulated log-score are expanded, so work scales with b_h rather than with the full ontology.

Evaluation follows the retrieval-and-ranking view. This thesis reports $\text{Hit}@k$ and $\text{nDCG}@k$ over $k \in \{1, 5, 10, 50, 100, 200, 500\}$ and trace speed–quality profiles across beam sizes and scoring modes, including head/tail breakdowns by label frequency. Models are trained on automatically labelled disease mentions derived from PubTator3, Wei et al. [2024] and CTD/MEDIC Davis et al. [2023, 2012] in two

Stage	Purpose	Operation / Learner	Inputs	Outputs
Representations	Shared space	TF-IDF $\oplus$ transformer, ℓ_2 norm	mentions; labels	$\phi(x)$; prototypes z_ℓ
Organisation	Compact regions	(Balanced) spherical k -means	Z	partition, ID maps
Routing	Where to look	OvR linear matcher	node $\phi(x)$; partition	child scores $\pi^{(h)}(x)$
Local ranking	Who wins	label-wise linear rankers	node $\phi(x)$; labels in child	label scores $r_\ell^{(h,c)}(x)$
Path context	Deeper cues	mean/sum/max over fused scores	node scores; positives	path-aware $\hat{z}_\ell^{(h)}$
Inference	Top- K list	beam search; fusion or cosine	query $\phi(x)$; hierarchy	ranked candidates

Table 1.1: XMR4EL pipeline stages and artefacts (cf. Figure 1.1).

snapshots—Disease-100 and Disease-500—that differ only in per-concept caps, following Ruas et al. [2024]; testing uses BC5CDR-Disease with MeSH normalisation Li et al. [2016]. Determinism is enforced end-to-end: randomness is fixed, learned partitions and ID mappings are persisted, and artefacts are versioned so runs are exactly repeatable.

1.4.1 Experimental Setup: Reproducibility and Hardware

Random seeds: Seeds are fixed by label-universe: **seed=0** for all 6k-label models and **seed=42** for all 1k-label models. Seeds are applied to Python, NumPy, and framework RNGs; any split/shuffling inherits the same seed per universe.

Top- k selection modes: Unless noted, experiments outside Hybrid X-Linker use `topk_mode=per_leaf`; Hybrid X-Linker uses `topk_mode=global`. The chosen mode is reported within each experiment.

Software stack: PyTorch 2.7.1 with the Metal (MPS) backend on Apple Silicon for 1k-label studies; CUDA on LASIGE servers for archival 6k-label runs. BLAS threads fixed via `OMP_NUM_THREADS` and `MKL_NUM_THREADS`.

Models and Hyperparameters: All the Models and hyperparameters used are pinned at (Appendix A).

Hardware: 6k (archival runs): LASIGE servers that contain $2 \times$ Intel Xeon Silver 4208 @ 2.10 GHz, 64 GB RAM with a single NVIDIA Tesla T4.

1k (latency/beam characterization): Apple **MacBook Air M3**, 16 GB unified memory, macOS [version], no discrete GPU; encoder executed with PyTorch MPS.

Comparability note: Headline quality metrics (e.g., Hit@100) are reported for the 6k-label universe. The 1k-label experiments (Section 3.9) are used to characterize beam-quality and latency; absolute Hit@ k differs mechanically with $|\mathcal{Y}|$ and is not compared directly to 6k.

1.5 Contributions

This thesis contributes (i) a *systemization* of XMR for biomedical entity spaces; (ii) a *domain adaptation* of PECOS-style pipelines to synonym-level, non-contextual entity normalization ; and (iii) an *empirical characterization* via an ablation-ready, reproducible codebase, publicly available at github.com/lasigeBioTM/XMR4EL.

The focus is on integration, engineering choices, and careful scoping rather than proposing new learning algorithms.

Delivered components.

1. **Modular, model-agnostic, open XMR framework for biomedical EL:** A configurable pipeline—*clustering* $\rightarrow$ *classifier-based routing* $\rightarrow$ *localized ranking*—with stable interfaces for encoders (sparse/dense), partitioners, matchers, losses, fusion rules, and evaluation. Components accept sparse/dense inputs and support plug-and-play experimentation under a common harness.
2. **Cluster-local per-entity OvR rankers with label-aware features:** After partitioning on PIFA prototypes, *lightweight* one-vs-rest rankers are trained *per label* within their cluster using a *label tile* (prototype concatenation) to model mention–label interactions. Labels with insufficient positives are skipped at training yet remain routable/scorable at inference via the matcher or prototype similarity (tail fallback).
3. **Curriculum hard-negative mining compatible with PECOS:** A two-phase regime: warm-start with mixed random and prototype-near negatives TFN, followed by cluster-local ANN/MIPS hard mining MAN with periodic refresh and diversity mixing. Mining is restricted to the label’s cluster to sharpen local decision boundaries.
4. **Hierarchical training with path-aware feature passing; optional rankers per layer:** Training repeats across depths $h \in \{0, \dots, H - 1\}$. Parent nodes emit compact score summaries that are injected as features into child nodes (for both mentions and labels). Rankers can be trained only at leaves or at every layer via configuration.
5. **Layer-wise beam routing with L_p fusion at inference:** Inference expands the top- b children per layer using matcher posteriors; at the leaf level, scores are combined via L_p interpolation (default $p=3$, $\alpha=0.5$). Routing is matcher-only. Final scoring is performed on labels within the surviving leaves.
6. **Sparse, memory-aware systems design:** Per-label models and inputs are stored in CSR/CSC; scoring uses sparse dot products; training supports label-parallelism and node-level batching. A custom balanced spherical k -means with size constraints is provided for partitioning (current initializer is non-deterministic and does not expose a seed).

7. **True open-source Hybrid X-Linker:** Made Ruas et al. [2024] compatible with this framework, enabling a truly open-source setup where users can swap in any model (clustering, matcher, rankers) they want to try. Publicly available at github.com/lasigeBioTM/X-Linker

Relation to prior work and scope: The architecture follows established PECOS/XMC patterns (partitioned routing, beam search, localized ranking). The contribution lies in the *integration and release* of a cluster-local OvR ranking stack with curriculum mining and path-aware feature passing for biomedical EL, together with a reproducible ablation framework.

1.6 Document Structure

Additionally to the present introductory chapter, this document is structured in four chapters as follows:

- **Chapter 2** (Related Work) introduces the basic concepts and resources that support XMR and EL.
- **Chapter 3** (XMR4EL) presents the work developed to create modular framework for XMR tailored to biomedical EL.
- **Chapter 4** (Conclusion) discusses the main conclusions of this work, and indicates some directions for future work.

Chapter 2

Related Work

This chapter surveys biomedical NEL at ontology scale and positions it within *extreme multi-label* methods. Section 2.1 formalizes NEL against MeSH/UMLS, motivates the standard two-stage design (fast generator → precision ranker), and lists canonical EL corpora with gold mention→ID supervision. Section 2.2 frames scalability as *eXtreme Multi-label* (XMC/XMR), emphasizing the organise→route→rank discipline and its principal families (OvR, trees/partitions, PLTs, graph retrievers, neural hybrids). Section 2.3 details sparse, dense, and hybrid representations for mentions and labels that shape partition quality and recall. Section 2.4 covers label-space organisation and routing policies (hard/beam, graph-based retrieval), while Section 2.5 presents label-wise ranking with a hard-negative curriculum for fidelity at leaf level. Section 2.6 consolidates evaluation and calibration conventions for XMR, XMC, and NEL. Finally, §**Hybrid X-Linker** instantiates the pipeline in biomedicine, combining abbreviation expansion, string matching, PECOS-style extreme ranking, and graph-based disambiguation, alongside weakly labelled training resources for scale.

2.1 Biomedical Named Entity Linking (NEL): Overview and Fundamentals

Imagine reading “*amyloidosis*”—a rare disease where amyloid proteins build up in organs—in a clinical note. A Named Entity Recognition (NER) system would spot the span and type it as a *disease*. A Named Entity Linking (NEL) system must go further: it must determine *which* disease that string refers to by grounding the mention to a canonical entry in a curated resource.

What is the system linking *to*? A Knowledge Base (KB) is a catalogue of entities with stable identifiers, names/synonyms, and relations; an *ontology* is the shared, machine-interpretable schema that specifies those concepts and relations so different systems agree on meaning Gruber [1993]. In biomedicine, this matters because the same disease can appear under many names across sources; linking to a shared identifier makes downstream search, analytics, and integration reliable. Table 2.1 summarises the two resources central to this work.

Two *general-purpose* resources are central in biomedical NEL. The Unified Medical Language System (UMLS) integrates dozens of terminologies and assigns each concept a Concept Unique Identifier (CUI) that unifies many synonymous names and records rich inter-concept relations Bodenreider [2004]. Medical Subject Headings (MeSH), maintained by the U.S. National Library of Medicine (NLM), provides a hierarchical vocabulary for indexing the literature; its descriptor IDs and tree numbers support structured retrieval and query expansion Lipscomb [2000].

In addition, this work uses two Knowledge Organization System (KOS) for disease and chemical linking: CTD MEDIC disease vocabulary (MEDIC) Davis et al. [2012] that merges MeSH and Online Mendelian Inheritance in Man (OMIM) disease terms for normalization and Comparative Toxicogenomics Database (CTD) Davis et al. [2023]-Chemical vocabulary aligned to MeSH Chemicals and enriched with synonyms and cross-references. In practice, identifiers in MEDIC and CTD-Chemical are largely MeSH-based, with CTD curation layered on top.

Resource	Maintainer	Identifier	Structure	Notes / Typical use
UMLS Bodenreider [2004]	NLM	CUI	Meta-ontology; integrates many vocabularies	Unifies synonyms; rich relations across sources.
MeSH Lipscomb [2000]	NLM	Descriptor ID	Curated hierarchical tree (tree numbers)	Indexing PubMed; hierarchy-aware retrieval.
MEDIC (CTD) Davis et al. [2012]	CTD	MeSH-based (merged)	Disease KOS merging MeSH+OMIM	Disease normalization in CTD; used by NCBI-Disease.
CTD-Chemical Davis et al. [2023]	CTD	MeSH-based	Chemical KOS aligned to MeSH	Chemical normalization; rich synonyms/cross-refs.

Table 2.1: Core biomedical KBs/ontologies and task-specific KOS targeted in this work.

Because UMLS and MeSH define a very large, alias-rich, hierarchical label space, two consequences follow: exhaustive per-mention scoring is infeasible, and literal matching alone is brittle.

Effective NEL systems therefore follow a two-stage recipe: a fast *generator* (lexical/BM25 retrieval, classifier-based routing, or bi-encoder retrieval) reduces millions of UMLS/MeSH entries to a small set; then a *ranker*—often a bi- or cross-encoder—scores mention–candidate pairs using richer signals Neumann et al. [2019].

Practical evaluation of these systems therefore relies on EL corpora that provide gold *mention* $\rightarrow$ *KB identifier* pairs.

MedMentions provides mention-level links to UMLS across thousands of PubMed abstracts and is often used in both its full form and the ST21pv subtype-restricted variant; it exposes broad ontology breadth and a heavy long tail Mohan and Li [2019]. *BC5CDR* comprises paired disease and chemical tracks with gold MeSH identifiers, enabling per-track EL as well as joint analyses; abstracts are annotated for both entity types Li et al. [2016]. *NCBI-Disease* offers curated disease normalization to MEDIC/MeSH with fine-grained distinctions and is widely used as a disease-only EL benchmark Doğan et al. [2014]. *NLM-Chem* supplies chemical mentions normalized to MeSH and serves as a standard chemical EL set Islamaj et al. [2021]. *BioRED* includes multiple entity types and relations at the document level; its disease/chemical subsets support EL while allowing evaluation of document-level coherence Luo et al. [2022]. Finally, *BELB* provides a normalized evaluation layer over multiple EL corpora (common pre-processing, metrics, and splits), and *BELHD* isolates “hard homonym” cases to stress genuine ambiguity

Garda et al. [2023]; Garda and Leser [2024]. All the key differences are described in the below table 2.2

Dataset / Suite	Entity types	Target KB(s)	Notes (NEL use; XMR-ready)
MedMentions	multiple (UMLS semantics)	UMLS	Broad coverage; mention-level EL; pronounced long tail.
BC5CDR – Disease	disease	MeSH	Standard disease EL; pairs with BC5CDR-Chemical.
BC5CDR – Chemical	chemical	MeSH	Standard chemical EL; pairs with BC5CDR-Disease.
NCBI-Disease	disease	MEDIC/MeSH	Curated disease normalization to KB IDs.
NLM-Chem	chemical	MeSH	Chemical EL benchmark with gold MeSH IDs.
BioRED (Disease/Chem)	disease, chemical	MeSH (subset)	Document-level context; EL labels per mention.
BELB (suite)	multiple	UMLS/MeSH (varies)	Normalized EL benchmarking suite (aggregates corpora).
BELHD (suite)	multiple	UMLS/MeSH (varies)	“Hard homonyms” EL subset from BELB.

Table 2.2: Biomedical EL corpora/suites (gold mention→ID). Each also supports XMR evaluation for candidate generation.

2.2 Extreme Multi Label Learning

The scale/skew constraints just outlined in the earlier section 2.1 are exactly what *extreme multi-label* methods are built to handle. We use XMC for the umbrella problem—selecting a relevant subset from a very large label universe—and we focus on its ranking-first specialization, XMR, where the deliverable is a high-recall *topk* candidate set computed under strict latency/memory budgets. This perspective motivates the organize→route→rank architecture that will be talked throughout this chapter and fixes the evaluation lens ($\text{Hit}@k/\text{nDCG}@k$ for candidates, with end-to-end EL reported separately).

Across the literature, “prune first, then rank” is implemented by a few recurring families. **Sparse OvR** keeps one-vs-rest but uses aggressive weight pruning and parallel CPU training (*DiSMEC*, *PPDSparse*)—accurate and interpretable, but typically paired with a prior pruning stage to avoid scoring all labels Babbar and Schölkopf [2017]; Jain et al. [2019]. **Label partitions/trees** (*Parabel*, *Bonsai*) organize labels with spherical clustering and route inputs through a shallow hierarchy, turning $\mathcal{O}(L)$ work into roughly $\mathcal{O}(\log L)$ decisions Prabhu et al. [2018]; Khandagale et al. [2020]; Chang et al. [2020]. Probabilistic Label Tree (PLTs) multiply node posteriors along a path for calibrated scoring, but are sensitive to under-confidence at any hop Jasinska et al. [2016]; Beygelzimer et al. [2014]. **Graph retrievers** (*AnnexML*, *SLICE*) replace rigid partitions with a k NN label graph and ANN search (often HNSW) to surface multiple routes to the same label Tagami [2017]; Jain et al. [2019]; Chang et al. [2020]. **Neural hybrids** (*X-BERT*, *X-XR-Transformer*) cluster labels semantically and fine-tune encoders on routed subsets, sometimes refining the tree as embeddings improve Yu et al. [2019]; Chang et al. [2020]; Zhang et al. [2021].

This “organize → route → rank” recipe is exactly what Prediction for Enormous and Correlated Output Spaces (PECOS)—systematizes Yu et al. [2022]. PECOS begins with *semantic label indexing* (sparse/dense/hybrid label vectors; then learns a *matcher*—a classifier-based router over a Parabel/Bonsai-style label tree—and uses beam search to route each input to a tiny, high-recall candidate slice; finally, it

Family / Examples	Train cost	Infer cost	Memory
Sparse OvR (DiSMEC/PPDSparse)	high (parallel)	$\mathcal{O}(\#\text{nz})$	med-high
Trees/Partitions (Parabel/Bonsai)	low-med	$\mathcal{O}(\log L)$ (beam)	low-med
Hier. linear (XR-Linear, PECOS)	med	$\mathcal{O}(\log L) + C $	low-med
Neural hybrids (X-/XR-Transformer)	high	$\mathcal{O}(\log L) + C $	med
Graph retrievers (AnnexML/SLICE)	med	ANN over labels	med

Table 2.3: Qualitative trade-offs across extreme families. $|C|$ denotes the routed candidate set size.

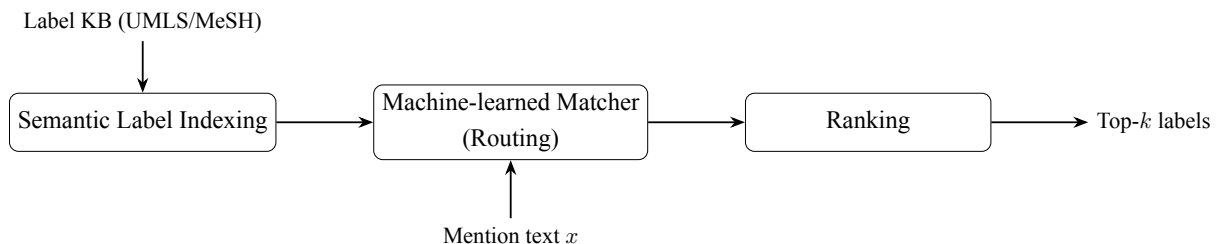


Figure 2.1: Minimal PECOS-style pipeline: index labels, route with a learned matcher, then rank candidates.

performs *ranking* only on that routed set with lightweight models (minimal pipeline can be seen in Figure 2.1). Two reference instantiations illustrate the trade-offs: *XR-Linear* learns linear routers and local linear rankers over the tree to deliver sub-millisecond candidate generation at scale, while *XR-Transformer* threads transformer encoders through the hierarchy and refines the tree as embeddings improve, boosting precision while keeping cost bounded Yu et al. [2022]; Zhang et al. [2021]. Read PECOS as an engineering playbook rather than a single model: it bakes “*prune first, then rank*” into the pipeline and sits alongside sparse OvR, PLTs, and graph retrieval in the same design space.

Having fixed the XMR framing and pipeline, the next question is what objects this machinery operates on. In particular, the geometry of *label indexing* and the behaviour of *routers* depend directly on how we vectorise mentions and labels. We therefore specify the sparse, dense, and hybrid representations used in this work next (Section 2.3), before turning to the construction of partitions and routing policies (Section 2.4), finalizing with specifics of rankers and negative mining (Section 2.5).

2.3 Representations for Mentions and Labels

Before we can scale anything, we need to decide *what* we are scaling. In biomedical NEL, the raw material is short, sometimes cryptic strings—“RA”, “amyloidosis”, “BRCA1”—and our first job is to turn those strings into vectors that capture what they mean. The simplest way is to look at the *tokens* themselves. With Term Frequency–Inverse Document Frequency (TF–IDF), we count how often a token appears

in a string and then down-weight tokens that are common everywhere (so “amyloidosis” matters more than “the”) [Ramos et al. \[2003\]](#). We do this not only for single words but also for *n*-grams: contiguous sequences of *n* words (to catch phrases like “cardiac infarction”) and sequences of *n* characters (to catch morphology, affixes, and even minor typos—handy for gene and chemical names). These TF–IDF vectors are *sparse*: most entries are zero, which makes scoring very fast because only non-zeros are touched.

For retrieval, a closely related family of methods has become the workhorse in search systems: Best Match 25 (BM25). Intuitively, BM25 balances the same three ingredients—term frequency, inverse document frequency, and document length—so that extra repetitions of a term help, but with *saturation* (the benefit tapers off), rare terms are boosted, and very long strings do not unfairly dominate [Robertson et al. \[2009\]](#). In practice, BM25 is a robust, transparent baseline for pulling likely candidates at ontology scale, and many NEL pipelines still use it to seed a shortlist efficiently.

Lexical cues get us far, but biomedicine is steeped in synonymy and paraphrase. Here, *embeddings*—numeric vectors designed to reflect meaning—shine. Modern embeddings come from *transformers*, models that use self-attention to let every token attend to every other token in context, producing a vector for the whole string [Vaswani et al. \[2017\]](#). Domain-adapted transformers such as Biomedical BERT (BioBERT) [Lee et al. \[2020\]](#) and PubMedBERT [Gu et al. \[2021\]](#) bring this machinery to biomedical text and routinely improve recall on paraphrases and rare terms. In downstream systems, these encoders are used either to *encode* mentions/labels into the *same* vector space for fast similarity search (often called bi-encoder retrieval), or to *re-score* specific mention–label pairs with richer interactions (cross-encoding) at higher cost. A complementary idea is synonym-aware pre-training: Synonym-aware pre-training of BERT (SapBERT) explicitly pulls UMLS synonyms together in embedding space so that different names for the same concept land near one another [Liu et al. \[2021\]](#).

Because sparse and dense signals excel at different things—sparse anchors are unbeatable for exact strings and symbols; dense vectors generalize across wording—many practical pipelines simply *combine* them. A hybrid vector might concatenate a TF–IDF block (optionally compressed with a low-rank projection) and a transformer embedding, then apply block-wise normalization so no single block dominates. This pattern recurs in extreme multilabel systems such as *X-/XR-Transformer* [Zhang et al. \[2021\]](#) and in modular toolkits like PECOS [Yu et al. \[2022\]](#) (already explained in Section 2.2), which expose both sparse and dense matchers. Hybrids tend to stabilize clustering and lift recall on tail concepts without sacrificing exact-match precision.

Labels need vectors, too. There are three common strategies. First, *instance-aggregated prototypes* (often called PIFA): average the vectors of all training mentions that link to a label; this ties label geometry to how terms are actually used in the corpus and is widely used when building label trees [Yu et al. \[2022\]](#). Second, *text-only encoders*: embed the label’s canonical name (and maybe a definition), which enables zero/few-shot behaviour for unseen labels but can drift from corpus usage. Third, *hybrid pseudo-documents*: concatenate canonical name + synonyms (and short glosses when available) and pass that through a transformer, capturing both lexical anchors and semantics at the cost of longer inputs. As with mentions, normalizing label vectors keeps cosine scores stable and search friendly.

Finally, a practical note about frequency. Biomedical label spaces are skewed (introduced in Section 2.1): a small head has many examples; a vast long tail has few. Prototypes for tail labels can be brittle. Two simple fixes help in practice: (i) smooth the prototype a little toward a global centroid so single-example labels are less extreme; and (ii) keep a hybrid representation where a low-rank lexical projection (truncated Singular Value Decomposition (SVD)) improves conditioning while the original sparse block preserves rare-token precision. Synonym-aware label encoders (e.g., SapBERT) also help tails by knitting variant names together even with limited supervision.

This ordering—strings $\rightarrow$ vectors (sparse, dense, or both) $\rightarrow$ label representations—is deliberate. In the next sections, when we cluster labels into trees, route mentions with a matcher, and rank a small candidate set, we are *operating on these vectors*. If a representation over-weights exact tokens, clusters will mirror token overlap and the router will behave like a lexical index; if it over-weights semantics, clusters will reflect synonymy at the risk of merging hard homonyms.

2.4 Partitioning and Routing: Organizing a Million Labels into a Few Good Options

Once mentions and labels live in a vector space, a practical question looms: how do we keep from scoring *every* label for *every* mention?

In extreme settings the answer is simple in spirit—first *organize* the label space, then *route* each query to a tiny, high-recall corner where fine discrimination actually matters.

Imagine all labels as points on a sphere: if we split them into coherent regions, a prediction no longer scans the horizon; it follows a small map. *Parabel* makes this concrete by recursively clustering labels with spherical k -means to build a balanced binary tree and training one-vs-rest models at the leaves, so depth grows like $\log_2 L$ and a query traverses only a handful of nodes [Prabhu et al. \[2018\]](#). *Bonsai* loosens those rules with k -ary splits and relaxed balance, yielding shallower, wider trees that curb error propagation at the cost of a larger fan-out per node [Khandagale et al. \[2020\]](#); [Chang et al. \[2020\]](#).

Partition quality is everything: good splits keep confusable labels together and far from the rest. Yet naïve spherical k -means can glue rare (tail) labels to frequent (head) labels simply because they co-occur, making later ranking harder; three practical tweaks help in practice—use *hybrid* representations (Section 2.3) so clusters reflect both lexical anchors and semantics, enforce gentle size constraints or rebalancing (Bonsai-style) to avoid tiny or overloaded children, and refine the tree as learning progresses so improved embeddings inform better splits in later rounds [Zhang et al. \[2021\]](#).

With a map in hand, you still need a travel policy: *hard routing* picks the single best child—fast but brittle if an early decision goes wrong—while *beam routing* keeps the top- B children per level and explores them in parallel; a modest beam (e.g., $B \in [10, 20]$) often recovers most recall for a small compute bump [Yu et al. \[2022\]](#).

A different philosophy, PLTs, assigns a probability to each child and scores a leaf by multiplying

probabilities along the path—elegant and calibrated when trained well, but sensitive: under-confidence at any node down-weights its entire subtree, and long paths can accumulate error [Jasinska et al. \[2016\]](#); [Beygelzimer et al. \[2014\]](#). And when one path is not enough, “soft partitions” step in: graph-based methods such as *AnnexML* and *SLICE* connect labels via a k NN graph and retrieve candidates by Approximate Nearest Neighbour (ANN) search—typically Hierarchical Navigable Small World (HNSW) index that lets queries hop coarse-to-fine layers to find near neighbours in sub-linear time with high recall—effectively giving multiple routes to the same label [Tagami \[2017\]](#); [Jain et al. \[2019\]](#); [Chang et al. \[2020\]](#). These “soft partitions” often achieve higher recall for ambiguous or multi-faceted labels and play well with lightweight re-rankers.

We can lean on existing hierarchies—MeSH trees and other ontologies—to seed or constrain learned partitions, but real ontologies are *polyhierarchical*: concepts often have multiple parents, so allowing *multi-membership* (placing a label in more than one cluster) reduces misrouting for ambiguous entities with negligible overhead in modern XMR systems.

Net effect: a good partition plus a sensible routing policy turns $\mathcal{O}(L)$ scoring into roughly $\mathcal{O}(\log L)$ decisions and a small candidate set with predictable latency and memory, and it hands the final stage—local rankers or a cross-encoder—a focused neighbourhood of genuinely confusable labels instead of the entire ontology.

2.5 Rankers and Negative Mining at Scale

Once routing has carved the ontology down to a tiny neighbourhood, the remaining job is classic learning-to-rank: make the true labels outrank the near misses [Liu et al. \[2009\]](#).

What determines whether rankers actually learn the fine distinctions is *which negatives they see*. Easy, random negatives mostly teach type boundaries; progress comes from *hard negatives*—semantically close impostors that the router also tends to surface.

Modern systems therefore adopt a *curriculum*: warm up with a mix of random and prototype-near negatives to stabilize training, then refresh with *hard* negatives mined by fast inner-product or ANN search (e.g., HNSW) *within the routed cluster*, so the model repeatedly confronts exactly the mistakes it is likely to make [Chang et al. \[2020\]](#).

This mirrors best practice in EL (hard negatives from top lexical/bi-encoder retrieval rather than uniform sampling) [Wu et al. \[2020\]](#) and meshes with the “prune first, then rank” discipline: routers tame *scale*, hard-negative curricula deliver *fidelity*, and the combination compounds gains without blowing up compute.

Concretely, the recipe I follow is pragmatic and reproducible: build hybrid representations so “hard” really means near in both lexical and semantic space, route with a modest beam so the neighbourhood is small but high-recall, warm-start rankers on mixed negatives so they do not collapse early, and then iterate hard-mining cycles restricted to the routed slice so training stays memory-aware and focused on true con-founders [Yu et al. \[2022\]](#); [Zhang et al. \[2021\]](#); [Chang et al. \[2020\]](#).

2.6 Evaluation and Calibration (NEL, XMC, and XMR)

Evaluation should correspond to the task definition.

When the problem is framed as XMC, the objective is to assess how well a system *orders* many plausible labels. Ranking-oriented metrics such as Precision at cut-off k ($P@k$) and Normalized Discounted Cumulative Gain at cut-off k ($nDCG@k$)—often with propensity-adjusted variants to reduce head–tail bias—are typically reported, together with head/mid/tail breakdowns so strong head performance does not mask long-tail failures.

Under the *ranking-first* stance of XMR, the deliverable is a high-recall *candidate set*. Consequently, $Hit@k$ (equivalently, $Recall@k$), Mean Reciprocal Rank (MRR), Mean Average Precision (MAP), and $nDCG@k$ become primary; these quantify an *upper bound* for any downstream decision module (assigner) Wu et al. [2020].

In NEL, the end-to-end question differs—whether the mention is linked to the correct identifier (ID). Common summaries include *accuracy* or *micro-F1/macro-F1* over mentions, sometimes with per-entity or per-type slices. The same two-stage logic applies: first report candidate-generation recall (XMR-style), then report final assignment quality (NEL-style) to localize errors Kolitsas et al. [2018]; Shen et al. [2014].

When scores from multiple stages must be fused (e.g., router *logits* combined with leaf-level ranker scores) or thresholds must be set (assign vs. abstain), *post-hoc calibration* helps make outputs comparable across branches and frequencies Leathart et al. [2017]. Typical choices include *Platt scaling* for sigmoid outputs and *temperature scaling* for softmax outputs. Pruning mistakes are irrecoverable; accordingly, high-recall candidate generation is a prerequisite for any calibrated assigner.

All key differences between tasks evaluations can be seen in an compact way in the table 2.4

Target	Metrics	Purpose / Notes
XMR (candidates)	$Hit@k$ ($Recall@k$), $nDCG@k$, MRR, MAP	Shortlist upper bound; report head/mid/tail; $k \in \{1, 5, 10, 50, 100\}$.
XMC (ranking)	$P@k$, $nDCG@k$ (propensity-adjusted when used)	Global ordering quality; mitigates head bias.
NEL (assignment)	Accuracy, micro-F1, macro-F1	End-to-end correctness; also report generator recall.
Calibration	ECE, NLL, Brier score	<i>Expected Calibration Error</i> (ECE), <i>Negative Log-Likelihood</i> (NLL); Probabilistic Label Trees (PLTs) may under-estimate on deep paths.
Efficiency	Latency, RAM, #labels scored	Record beam size (in beam search), $ C $ (routed candidate set size), and batch settings.

Table 2.4: Compact evaluation map for NEL, XMC and XMR.

Two conventions improve comparability across systems. First, it is useful to *pair* a candidate-generation curve ($Hit@k/nDCG@k$ as a function of *beam size* and per-leaf k) with the end-to-end NEL metric, so bottlenecks in routing or ranking are visible; this also quantifies the *gap* between the XMR upper bound and realized assignment. Second, results should be *stratified* by frequency (head/mid/tail) and—given biomedical ontologies—by ambiguity (homonym-heavy vs. synonym-heavy mentions): gains that help the head can leave the tail unchanged, while synonym-aware encoders often excel on paraphrases yet re-

main brittle on true homonyms [Garda and Leser \[2024\]](#); [Sung et al. \[2020\]](#). Finally, reports of calibrated assign/abstain behaviour should include Expected Calibration Error (ECE) or Negative Log-Likelihood (NLL) *before and after* calibration and should state clearly which scores were calibrated (router, ranker, or fused), since mixing calibrated and uncalibrated streams can appear strong on $P@k$ yet behave poorly once thresholds matter in deployment [Leathart et al. \[2017\]](#).

2.7 Hybrid X-linker

Hybrid X-Linker is a biomedical entity linking EL system that links disease and chemical mentions to controlled vocabularies—MEDIC, [Davis et al. \[2012\]](#) for diseases and CTD-Chemical [Davis et al. \[2023\]](#), for chemicals—through a modular, hybrid pipeline [Ruas et al. \[2024\]](#). The pipeline progresses from abbreviation expansion (mapping short forms like *COPD* to *Chronic Obstructive Pulmonary Disease*) to a string matcher that retrieves candidate IDs via exact or fuzzy matches against knowledge-base names and synonyms, and then to an extreme multi-label ranking component (PECOS-EL) that scores candidate concept IDs directly from the mention string. Candidates from the matcher and PECOS-EL are filtered and merged with simple rules—accepting perfect-score matches immediately, otherwise applying a score threshold and preserving diversity—and the resulting per-mention candidate lists are fed into a global disambiguation step that runs Personalized PageRank (PPR) over a graph built from ontology relations. This final step promotes document-level coherence among candidates for different mentions. Notably, both Hybrid X-Linker and the SapBERT baseline operate primarily on the mention string (rather than full sentential context); thus, the graph step is the main mechanism for injecting document-level evidence [Ruas et al. \[2024\]](#). Figure 2.2 describes all the steps the model takes until the end of the inference step, returning the KB IDs.

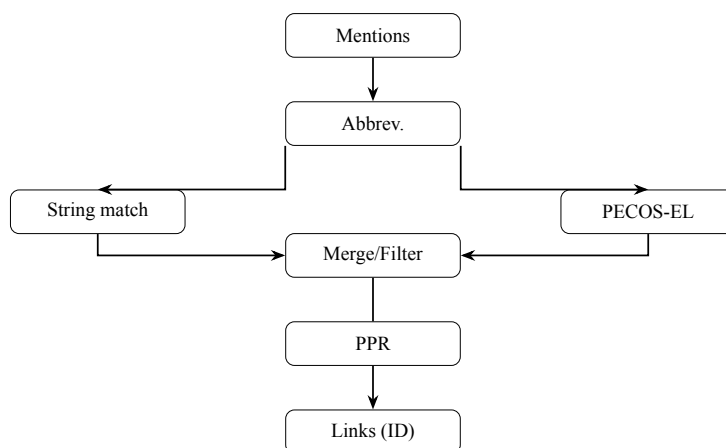


Figure 2.2: Hybrid X-Linker (inference): Mentions $\rightarrow$ Abbrev. $\rightarrow$ {String match || PECOS-EL} $\rightarrow$ Merge/Filter $\rightarrow$ PPR $\rightarrow$ KB IDs.

A central contribution is the automatic construction of large, weakly labeled training corpora for PECOS-EL. The authors first assemble KOS-only baselines—**Disease-KOS** and **Chemical-KOS**—containing all canonical names and synonyms from MEDIC and CTD-Chemical. They then harvest PubTator3 annotations from PubMed and map each mention to a valid MeSH-linked KB identifier, yielding millions of mention $\rightarrow$ ID pairs Ruas et al. [2024]. For chemicals, this pool is tractable as a single **Chemical-All** file (KOS labels plus all PubTator chemical mentions). For diseases, the pool is much larger, so they release progressively capped variants that limit the number of PubTator-derived instances per concept while retaining all KOS synonyms: **Disease-100** ($\sim 828,163$ instances), **Disease-200** ($\sim 1,402,332$), **Disease-300** ($\sim 1,873,901$), **Disease-400** ($\sim 2,275,258$), alongside **Disease-KOS** (89,465) and the uncapped **Disease-All** ($\sim 9,497,985$) Ruas et al. [2024] (All the different datasets sizes are showed in table 2.5). Although not discussed in the paper text, the public release also includes larger caps such as **Disease-500** (and even **Disease-1000**); these follow the same two-column, line-based format (integer concept index and associated string) and are generated by increasing the per-concept cap in the PubTator-derived pool.

Type	Name	#Instances
Chemical	Chemical-KOS	452,318
Chemical	Chemical-All (KOS + PubTator)	1,123,842
Disease	Disease-KOS	89,465
Disease	Disease-100 (KOS + PubTator, cap 100)	828,163
Disease	Disease-200 (cap 200)	1,402,332
Disease	Disease-300 (cap 300)	1,873,901
Disease	Disease-400 (cap 400)	2,275,258
Disease	Disease-All (no cap)	9,497,985

Table 2.5: Training-file variants and sizes reported for Hybrid X-Linker.

For evaluation, Hybrid X-Linker is tested on standard corpora—BC5CDR (disease and chemical) Li et al. [2016], NCBI-Disease, BioRED (disease and chemical subsets) Luo et al. [2022], and NLM-Chem—using the usual filtering of nil or obsolete identifiers Ruas et al. [2024]. Information about these datasets can be see in the table 2.2.

In my experiments I adopt **Disease-100** as the disease training file; the public artifacts also expose larger disease caps such as **Disease-500**, which, while not analysed in the paper, are available for training the PECOS-EL disease model with increased data scale.

Ablations reported in Ruas et al. [2024] show that abbreviation detection yields substantial gains, and that adding the PageRank-based global disambiguation on top of string matching plus PECOS-EL produces the strongest overall results. Across benchmarks, Hybrid X-Linker attains competitive top-1 accuracy—around 0.83 on BC5CDR-Disease—and surpasses SapBERT on roughly half of the evalua-

tions (e.g., BC5CDR-Disease, NCBI-Disease, BioRED-Chemical), with SapBERT stronger on the others. Together, these findings support the utility of combining knowledge-based string signals, scalable extreme ranking, and ontology-driven coherence when manual EL supervision is scarce.

Chapter 3

eXtreme Multi Label Ranking for Entity linking (XMR4EL)

This chapter deepens the methodology from §1.4 with system details: representations and supervision (§3.1), label-space organisation (§3.2), classifier-based routing (§3.3), label-wise ranking (§3.4), hierarchical composition (§3.5), inference (§3.6), and systems considerations (§3.7).

Model-agnostic design. XMR4EL is *model-agnostic*: components expose stable interfaces (inputs/outputs, artefact formats), so vectorizers, transformers, clustering models, matchers and rankers can be swapped without changing data handling or evaluation.

3.1 Representations and supervision

The pipeline begins with the controlled vocabulary: each concept comes with a list of surface forms, and the first task is to turn those lists into supervision and vectors that later components can reason over. A global label order $1:L$ is fixed once and persisted, together with $label_id \leftrightarrow KOS\ identifier$ maps, so every downstream artefact speaks the same indexing language. Synonym lists are then flattened into individual training mentions and a reverse map (label $\rightarrow$ mention-row IDs) is written out, ensuring that any future rebuild can trace exactly which rows correspond to which labels. From here, the path from strings to matrices is entirely deterministic.

Text is lightly but deliberately normalised. Unicode is canonicalised (Normalization Form Canonical Composition (NFC)/Normalization Form Compatibility Composition (NFKC)), repeated whitespace collapsed, and trivial spacing artefacts repaired; however, case is *not* forced to lower because it can carry meaning in biomedical strings, and punctuation is *not* stripped for the same reason. Stopwords are retained, as they sometimes anchor idioms (e.g., “*disease of X*”). Duplicate forms within a label’s own synonym set are removed to avoid overweighting. Medically meaningful symbols—Greek letters, hyphens—are preserved after canonicalisation. Before any vectors are built, obsolete or redirected KOS entries are filtered or mapped so that supervision does not leak across vocabulary snapshots.

Mentions and labels must live in a common space if retrieval and ranking are to be fast and stable. Each mention x is mapped by concatenating a **sparse lexical** block with a light **transformer** block and then enforcing unit norm,

$$\phi(x) = [\lambda_{\text{lex}} \hat{\phi}_{\text{tfidf}}(x) ; \lambda_{\text{ctx}} \hat{\phi}_{\text{tr}}(x)], \quad \|\phi(x)\|_2 = 1,$$

where the hats denote *block-wise* ℓ_2 normalisation so neither block dominates by scale. The lexical side typically mixes token n -grams with a small band of character n -grams to stay robust to orthographic variants; the transformer side yields a fixed-size sentence embedding directly from the string (no document context). The weights $\lambda_{\text{lex}}, \lambda_{\text{ctx}} \geq 0$ balance sparse vs contextual cues (defaults $\lambda_{\text{lex}} = \lambda_{\text{ctx}} = 1$, tuned on a development split when available). After concatenation, a final ℓ_2 normalisation makes cosine a pure dot product.

Memory and conditioning are controlled before anything grows too large. If needed, truncated SVD is applied to the *lexical* block only (rank $r \ll d_{\text{lex}}$) to compact redundant dimensions while keeping the contextual block at its native size. The transformer block is then *sparsified* at export—either by magnitude thresholding or by keeping the top- t absolute components per row—so that the unified feature space remains sparse end-to-end. In practice this yields substantial savings with negligible impact on retrieval quality (see §3.7). Stacking the normalised mention vectors produces $X \in \mathbb{R}^{n \times d}$ with unit-norm rows.

Supervision is encoded as an incidence matrix $Y \in \{0, 1\}^{n \times L}$, marking which labels are positives for which mention rows. In synonym-derived settings, rows are typically single-hot, but the multi-hot form allows for cross-vocabulary merges when the KOS dictates them. No long-tail pruning is applied here; rare labels are kept and allowed to find their footing downstream.

Labels themselves are given geometry via *PIFA* computed strictly on *training* positives to avoid leakage:

$$z_\ell = \frac{\sum_{i: Y_{i\ell}=1} X_{i:}}{\left\| \sum_{i: Y_{i\ell}=1} X_{i:} \right\|_2 + \varepsilon}, \quad \varepsilon = 10^{-12},$$

yielding $Z \in \mathbb{R}^{L \times d}$ with unit-norm rows. Because X is block-normalised before aggregation, neither the lexical nor the contextual block overwhelms the prototype geometry. With mentions and labels now co-habiting a single, sparse, cosine-friendly space, later stages can treat candidate generation and ranking as cheap inner products.

Before any artefact leaves this stage, a few invariants are enforced. Every label must have at least one positive; rows of X and Z must be unit norm within tolerance; Y must have no all-zero columns; the persisted label order must agree across all files; and the reverse label $\leftrightarrow$ mention map must reconcile with Y . Any violation halts the run with a clear diagnostic—better to fail fast than to hide a silent drift.

The outcome is a compact bundle that other components can rely on: the mention matrix X (CSR/CSC), supervision Y (CSR/CSC), the prototype matrix Z (CSR/CSC), and the immutable label $\leftrightarrow$ mention map

(dictionary/Parquet). Alongside these, the normalisation settings, feature vocabularies, SVD rank, sparsification policy, and the weights $(\lambda_{\text{lex}}, \lambda_{\text{ctx}})$ are logged and the artefacts are checksummed so that a run can be reproduced byte-for-byte. A concise summary of shapes, sparsity and storage is given in Table 3.1.

Artefact	Symbol	Shape	Sparsity	Storage	Notes
Mention embeddings	X	$n \times d$	high (CSR)	CSR/CSC	by TF-IDF $\oplus$ sparsified transformer; rows ℓ_2 -norm
Supervision	Y	$n \times L$	very high	CSR/CSC	Multi-hot; typically single-label per row
Label prototypes (PIFA)	Z	$L \times d$	med (CSR)	CSR/CSC	From positives; rows ℓ_2 -norm
Label $\leftrightarrow$ mention map	—	—	—	dict / parquet	Deterministic reverse map for reproducibility
Lexical block	$\hat{\phi}_{\text{tfidf}}$	$n \times d_{\text{lex}}$	very high	CSR/CSC	Optional SVD to rank $r \ll d_{\text{lex}}$
Transformer block (pre-sparsify)	$\hat{\phi}_{\text{tr}}$	$n \times d_{\text{ctx}}$	dense	dense	Unit-norm per block before concat
Transformer block (stored)	$\hat{\phi}_{\text{tr}}^{\text{sp}}$	$n \times d_{\text{ctx}}$	high	CSR/CSC	Magnitude thresholding / top- t per row

Table 3.1: Artefacts produced by representations and supervision. Shapes, sparsity, and storage formats are explicit; mention and label vectors are unit norm. Defaults: TF-IDF if unspecified; BioBERT as the transformer if unspecified.

3.2 Label-space organisation

Label-space organisation partitions the unit-normalised prototypes Z under cosine geometry into compact regions that localise genuine confusions and bound candidate sets.

By default, a balanced spherical k -means encourages roughly equal child sizes subject to a minimum-membership threshold τ ; when balance proves infeasible or strict determinism is required, a seeded spherical k -means with k -means++ initialisation and `random_state=0` is used instead. The resulting single-membership assignment is stored as a sparse one-hot matrix $C \in \{0, 1\}^{L \times K}$ whose rows sum to one (exactly one child per label at a given depth). The choice of K targets compact, high-recall neighbourhoods selected on development data to maximise $\text{Hit}@k$ under a beam budget, with typical per-node $K \in [6, 12]$ and optional bounds on child size to stabilise routing. After a split, clusters smaller than τ are pruned without reassignment to preserve a stable topology; if pruning leaves fewer than two viable children, the split is deemed to have failed and training for that configuration halts with a diagnostic (fail-fast). Learned partitions and ID maps are persisted and reused across training and evaluation so that routing paths remain byte-for-byte repeatable. A compact view of the clustering objective and single-membership layout appears in Figure 3.1.

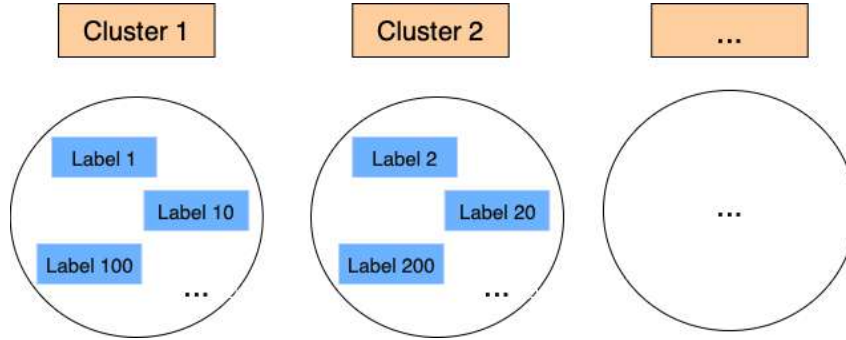


Figure 3.1: Label-space organisation: labels are distributed *through* clusters under cosine geometry.

3.3 Matcher (classifier-based routing)

The matcher is a multi-class classifier over the K clusters: for a unit-norm mention vector x (a row of $X \in \mathbb{R}^{n \times d}$), it predicts *which cluster contains the mention's gold label*. Let $C \in \{0, 1\}^{L \times K}$ be the single-membership label $\rightarrow$ cluster map (each label appears in exactly one cluster). Training keeps only mentions that have at least one positive among the labels, forms the restricted incidence Y , and induces cluster targets by projecting labels onto clusters,

$$M = \mathbb{I}[Y C > 0] \in \{0, 1\}^{n \times K},$$

so, under the synonym-derived single-label setting used here together with single-membership labels, each row of M has exactly one 1 (the remaining $K - 1$ entries are zeros).

By default, the learner is a light one-vs-rest *linear* logistic classifier with ℓ_2 regularisation (SGDClassifier, early stopping, max_iter=1000, random_state=0) trained on ℓ_2 -normalised, float32 features. For each cluster k , an independent head returns $\pi_k(x) = \sigma(w_k^\top x + b_k) \in (0, 1)$. Scores are not normalised across k (no softmax); only their relative magnitudes are used by downstream consumers. *Any linear OvR classifier* compatible with the same interface can be used interchangeably (e.g., liblinear logistic, linear Support Vector Machine (SVM) with probability calibration), and artefact formats remain unchanged. The setup is illustrated in Figure 3.2 and defaults are summarised in Table 3.2.

Edge cases and checks are handled explicitly. If a column of M is all zeros (no positive mentions for a cluster)—which should not occur with a sound partition—the corresponding head is skipped and a warning is logged; otherwise every retained cluster has an OvR head. Shapes are validated to ensure M and C are compatible and that no trained cluster is empty.

3.4 Label-wise ranking

Within each cluster, a family of *label-specific* binary rankers—one per resident label—separates near neighbours at fine granularity. Using the cluster targets $M = \mathbb{I}[Y C > 0] \in \{0, 1\}^{n \times K}$, the candidate

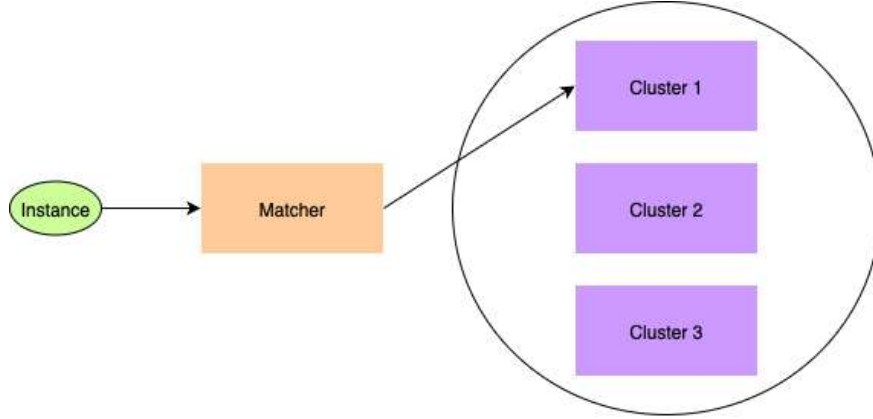


Figure 3.2: Classifier-based routing over K clusters: independent sigmoid heads yield non-normalised scores $\pi(x)$ per cluster; only their relative magnitudes are used downstream.

Setting	Default
Learner	OvR linear logistic (<code>SGDClassifier</code>)
Regularisation	ℓ_2 penalty; early stopping; <code>max_iter</code> =1000
Inputs	ℓ_2 -normalised sparse X , <code>float32</code>
Targets	$M = \mathbb{I}[YC > 0] \in \{0, 1\}^{n \times K}$ (one-hot)
Outputs	Independent sigmoids $\pi_k(x) \in (0, 1)$; no softmax
Determinism	<code>random_state</code> =0; artefacts persisted

Table 3.2: Matcher configuration and operational defaults.

mentions for cluster c are $\mathcal{I}_c = \{i : M_{ic} = 1\}$. For a label ℓ hosted by c , positives are $\{i \in \mathcal{I}_c : Y_{i\ell} = 1\}$ and negatives are $\mathcal{I}_c \setminus \text{positives}$. At least two positives and two negatives are required; otherwise training for ℓ is skipped (the label remains routable by other components). Figure 3.3 sketches the per-label view.

The ranker learns best by repeatedly meeting the mistakes it is likely to make. A compact two-phase curriculum mirrors PECOS Yu et al. [2022]: *Teacher-Forcing Negatives (TFN)*—identified using the ground-truth cluster indicator m_i —and *Matcher-Aware Negatives (MAN)*—identified using the matcher’s predicted cluster indicator $\hat{m}_i$. TFN aligns with what is semantically confusable; MAN aligns with what the router actually tends to surface; the union is empirically strongest. In the implementation this appears as **MTFN** $\rightarrow$ **MMAN**. The warm start **MTFN** blends easy and near negatives inside c —about 70% random/in-batch and 30% prototype-near (by cosine to z_ℓ)—to stabilise early updates without depending on matcher predictions. After warming up, **MMAN** refreshes, at each epoch, a hard pool the matcher is likely to co-surface: roughly 60% mined by fast inner-product/ANN search within c , 30% prototype-near, and 10% random to preserve diversity. Per-label pools are capped

$$N_{\text{neg}} = \min\{\text{neg } n_+, n_-\},$$

where neg is a number between 1 and the max of `int32` chosen by the user, by default is set to 15; n_+ and n_- are the counts of available positives/negatives for ℓ in c ; pools are deduplicated, rebuilt once per epoch, and if a slice becomes single-class the epoch for ℓ is skipped.

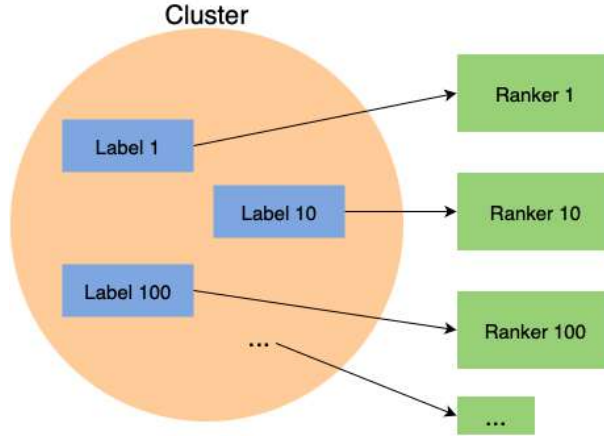


Figure 3.3: Label-wise ranking: one binary model per resident label separates confusable neighbours within a cluster.

Setting	Default
Scope	One-vs-rest, per-label binary ranker within cluster c
Learner	Linear hinge (<code>SGDClassifier</code>) with ℓ_2 penalty
Training	Incremental <code>partial_fit</code> ; early stopping; <code>max_iter</code> =1000
Inputs	Concatenated $[x_i; z_\ell]$ (mention; label prototype), row-wise ℓ_2 -normalised, <code>float32</code>
Targets	$y_{i\ell} \in \{0, 1\}$ for $i \in \mathcal{I}_c$; require ≥ 2 positives and ≥ 2 negatives, else skipped
Negatives (curriculum)	MTFN → MMAN ; warm: 0.70 random/in-batch + 0.30 prototype-near; hard: 0.60 ANN-mined + 0.30 prototype-near + 0.10 random; per-epoch refresh, dedup; $N_{\text{neg}} = \min\{neg \cdot n_+, n_-\}$ with default neg =15
Outputs	Real-valued margins $s_\ell(x)$ (no softmax/sigmoid by default)
Determinism & persistence	<code>random_state</code> =0; fixed per-epoch seed; persist label→ranker map and curriculum config
Option	MMAN-only schedule (skip warm start) once routing posteriors are reliable

Table 3.3: Label-wise ranker configuration and operational defaults.

Because the negative set evolves over epochs, models are trained *incrementally* with `partial_fit` (fixed seed) rather than re-initialised at each refresh. To encode mention–label interactions directly, each training vector tiles the label prototype alongside the mention features, $[x_i; z_\ell]$, followed by row-wise ℓ_2 renormalisation so margins remain comparable across labels. The default, the reference learner is a sparse, margin-based *linear* classifier with hinge loss and ℓ_2 regularisation (`SGDClassifier`, early stop-

ping, max_iter=1000, random_state=0, float32). Class weighting is not applied here, as balance is governed by the sampling curriculum itself. For speed-sensitive regimes a **MMAN-only** schedule is available (skipping the warm start once routing posteriors are reliable). For reproducibility and ablations, the system persists, per cluster, the mapping from global label IDs to trained rankers, the curriculum configuration (warm-up length, mix ratios, refresh cadence, neg_mult=neg, seed). The setup is illustrated in Figure 3.3 and defaults are summarised in Table 3.3.

3.5 Hierarchical training

This trio is made into a multi-layer pipeline by applying *label-space organisation* (§3.2), the *matcher* (§3.3), and *label-wise ranking* (§3.4) repeatedly over depths $h = 0, \dots, H-1$. At depth h , a node consumes four aligned artefacts— $X^{(h)}$ (mentions restricted to the node’s labels), $Y^{(h)}$ (local incidence), $Z^{(h)}$ (label embeddings in the same space), and deterministic local $\leftrightarrow$ global ID maps—and runs the same three steps as in the single layer (see Fig. 3.4 for a schematic).

The *organisation* step at depth h yields a single-membership assignment $C^{(h)}$; the same fail-fast policy as in §3.2 applies—if pruning leaves fewer than two viable children, the split is rejected and depending on the depth, it can happen two things: at depth $h = 0$ the training for that configuration halts with a diagnostic; at $h > 0$ the layer where the rejection is made, is discarded, and the model saves it itself with the previous layers. This is made to not create an fragile partition. Given $C^{(h)}$, *matcher* targets are induced as $M^{(h)} = \mathbb{I}[Y^{(h)}C^{(h)} > 0]$ (one-hot under the single-label, single-membership setting), and the reference linear OvR matcher is fitted exactly as described in §3.3. Within each retained child, *label-wise rankers* are trained as in §3.4 (same curriculum, features $[x; z_\ell]$, and incremental updates), unless disabled for speed-sensitive runs.

What is specific to the hierarchy is the *score propagation* used to prepare the next depth. After a node is trained, its scores are distilled into compact summaries and appended before re-normalisation, so deeper splits “see” upstream evidence without inflating dimensionality. On the *mention side*, for each kept child c three scalars are formed from the parent’s outputs for a query row: the score into c , the row-sum over all children, and the row-max; this 3-vector is concatenated to the mention row of $X^{(h)}$ and ℓ_2 re-normalised to yield $X_c^{(h+1)}$. On the *label side*, for each label ℓ resident in c , three statistics are computed between the child’s mentions and the base prototype z_ℓ —mean, sum, and max of mention–prototype inner products—and concatenated to z_ℓ before re-normalising to yield $Z_c^{(h+1)}$. These six numbers (three per side) act as a stable, low-dimensional “path context” that sharpens later clustering and ranking without changing the ambient embedding size.

Children are then instantiated by slicing artefacts per retained c : $Z^{(h)}$ is restricted to the child’s labels (with fresh local/global maps), and only mentions with at least one positive in that subset are kept to form $(X_c^{(h)}, Y_c^{(h)})$. Optional depth-wise knobs (e.g., reducing K_h via cut_half_cluster) control branching as depth increases. The process repeats for $h+1$ using $(X_c^{(h+1)}, Y_c^{(h+1)}, Z_c^{(h+1)})$.

For reproducibility, every trained component is persisted: the partition $C^{(h)}$, matcher parameters and

metadata, ranker inventories (when enabled), the score summaries used for propagation, node-local embeddings, and deterministic parent-child index maps. The resulting layered artefact $\mathcal{H} = [\mathcal{L}_0, \dots, \mathcal{L}_{H-1}]$ records both parameters and topology so that any reference to “child c at depth h ” is byte-for-byte identical across runs (Fig. 3.4).

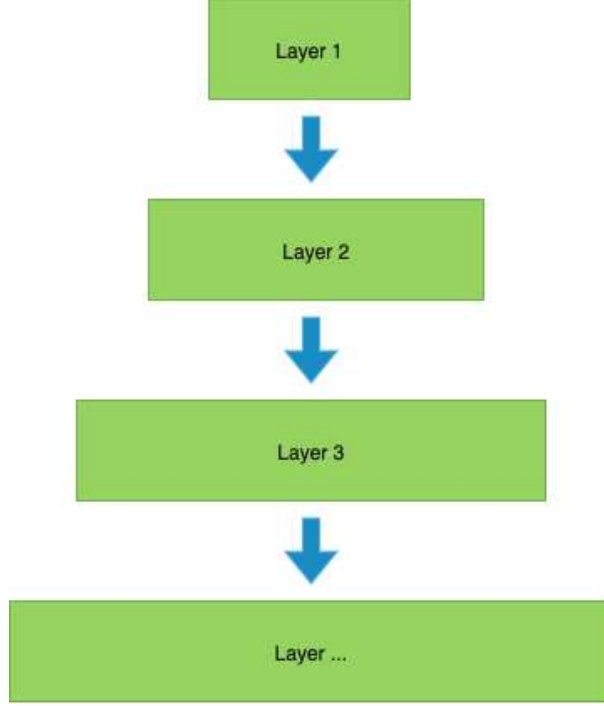


Figure 3.4: Multi-layer training: the organise $\rightarrow$ match $\rightarrow$ rank trio is applied by depth; compact mention- and label-side score summaries are concatenated and re-normalised to prepare artefacts for the next depth.

3.6 Inference

At test time a query string x_q is encoded once into a unit vector $v = \phi(x_q)$ and routed coarse-to-fine with a budgeted expansion. The traversal maintains a set of active states; at a non-terminal depth h , each active state produces child scores $\pi^{(h)}(v) \in \mathbb{R}_{\geq 0}^{K_h}$ from its matcher. The top $B^{(h)}$ children by accumulated path score are retained. For each retained child c the feature row is augmented in the same way as during training: three scalars $[\pi_c^{(h)}(v), \sum_j \pi_j^{(h)}(v), \max_j \pi_j^{(h)}(v)]$ are concatenated and the row is re-normalised to unit length; the path score is updated by $\log(\max\{\pi_c^{(h)}(v), \varepsilon\})$. All proposed children across current states are pooled, ranked by the updated path score, and pruned back to the top $B^{(h)}$ before descending. After the final expansion, the active set consists of visited leaves together with their query-specific, augmented feature rows. The process is illustrated in Figure 3.5.

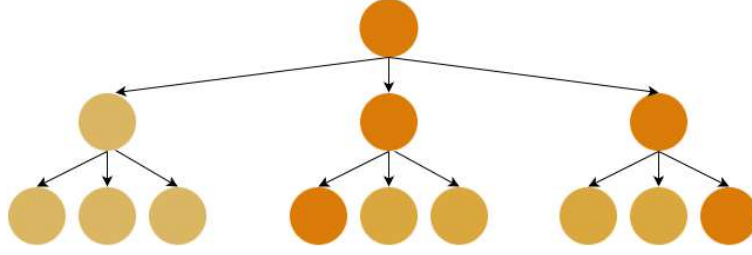


Figure 3.5: Inference with budgeted expansion (shown with $B(h) = 2$ for clarity): at each depth, only the highest-scoring branches are expanded; candidates are taken from visited leaves and scored by per-leaf fusion or global cosine. Nodes with darker colours are those traversed by the mention.

Inside a visited leaf, scoring follows two interchangeable modes. *Per-leaf fusion* mirrors the training interface: a small leaf matcher proposes up to k_{leaf} leaf-clusters per query (if present), each expands to its resident labels, and every candidate with global id g (leaf-local index ℓ in cluster c) receives a matcher score m and a label-evidence score r —either the probability from a trained label-wise ranker applied to the label-aware features $[x^{(H)}; z_g]$, or the constant 1 if no ranker is present. A numerically stable blend then produces the per-label score

$$s_g = \begin{cases} \left((1 - \alpha) m^p + \alpha r^p \right)^{1/p} & (\text{lp_fusion, default } p=3, \alpha=0.5) \\ m^{1-\alpha} r^\alpha & (\text{geometric}), \end{cases}$$

with m, r clipped to $[\varepsilon, 1]$ before fusion. Candidates may be truncated within each leaf to the top k_{leaf} by s_g , or all scored labels may be retained. Across all visited leaves, the union $\mathcal{C}_q$ of global ids is formed; if a label appears via multiple paths, the maximum s_g it achieved is kept.

In the *global-cosine* mode, a small number of candidates are gathered from each visited leaf (e.g., up to k_{leaf} per leaf) and the union $\mathcal{C}_q$ is re-ranked in the shared embedding space by

$$s_g = \langle v, Z_{g:} \rangle, \quad \|v\|_2 = \|Z_{g:}\|_2 = 1,$$

selecting the global top- K (or returning the full ranked list if no truncation is requested). This mode vectorises cleanly as a matrix–vector product over the candidate rows of Z .

Outputs are returned in two forms. First, a structured paths object records the traversal: at each depth, which children were expanded, their local scores $\pi^{(h)}(\cdot)$, and the accumulated path scores (useful for introspection and ablation). Second, a sparse row $S \in \mathbb{R}^{1 \times L}$ contains non-zeros only at the candidate labels—either the fused per-leaf scores or the cosine similarities over $\mathcal{C}_q$ in global-cosine mode.

Costs are predictable. Selection replaces $\mathcal{O}(L)$ work with $\sum_h \mathcal{O}(B^{(h)} K_h)$ to choose children plus scoring over a small candidate set; in global-cosine mode, final scoring costs $\mathcal{O}(|\mathcal{C}_q| d)$ per query and is typically more stable than evaluating $|\mathcal{C}_q|$ independent label models. Memory is dominated by the label embedding matrix Z and any materialised leaf rankers; latency is dominated by matcher evaluations

along the traversal and, when fusion is enabled, by the number of surviving label-wise evaluations. Ties in child scores are resolved deterministically to keep traversals repeatable.

3.7 Efficiency, memory, and systems considerations

Scaling to millions of labels is as much a systems exercise as an algorithmic one, so the design targets throughput, memory, and reproducibility from the outset (all stochastic knobs fixed; `random_state=0`). The governing idea is simple: keep representations and data *sparse-first*, batch work where topology allows, cache aggressively, and let routing cap the candidate set so later scoring stays predictable.

Sparse-by-default data and models. Inputs and intermediate matrices are stored in CSR/CSC, large arrays are memory-mapped, and label embeddings Z live in `float32` and are pre-normalised ($\|Z_g\|_2 = 1$) so cosine reduces to a dot product. With sparse X and ℓ_1 -regularised linear learners, many coefficients are exactly zero, so scoring reduces to inner products over active coordinates; cost scales in $\text{nnz}(x)$ (and the effective support of the model) rather than in the ambient dimension, echoing DiSMEC/Parabel-style gains but applied locally within clusters. *Disclaimer:* while the **reference configuration uses sparse linear models** (fast on CPUs and memory-light), the component interfaces are model-agnostic—users may choose **dense** learners (e.g., dense logistic, Multilayer Perceptron (MLP) on GPU). In that case, compute/memory trade-offs shift accordingly, but artefact formats and evaluation remain unchanged.

Batching and parallelism. Because different queries traverse different branches, batching respects topology. During training, node/cluster-level *branch-parallelism* evaluates matchers with vectorised sparse kernels (or dense General Matrix Vector Multiply (GMEV) if a block is dense); rankers train independently per label (*label-parallelism*); and matchers admit *data-parallelism* (mini-batches on GPU for dense encoders, multi-threaded CPU for linear). At inference, after routing fixes visited leaves, candidate labels are independent and rankers (when enabled) are evaluated concurrently. CPUs (or specialised sparse kernels) tend to dominate irregular sparse dot products; GPUs help for dense embedding, ANN/MIPS hard-mining, or fast cosine re-ranking across large candidate blocks.

Footprint control. Three choices keep memory in check: (i) ℓ_1 (optionally group-sparse) regularisation and feature selection in linear learners; (ii) truncated SVD for the lexical block plus a fixed sparse projection for the transformer block so the unified space stays light; and (iii) strictly *cluster-local* rankers so each label model only learns to discriminate among near neighbours. Large artefacts are memory-mapped to avoid ballooning resident memory.

Caching and reuse. Per-mention representations (sparse TF-IDF and projected transformer features) are computed once and reused in both matcher and rankers; label embeddings are persisted for fast retrieval. Intermediate products (e.g., layer-wise transformer outputs if enabled) and routing traces are cached so diagnostics and ablations (e.g., different top- K cut-offs) can be recomputed without repeating full traversals.

Ensembles with restraint. Multiple partitions/trees can improve robustness but multiply compute. Since per-label rankers already specialise locally, modest *matcher ensembles*—union of candidates from

alternative partitions—are preferred to raise recall at small extra cost while leaving leaf scoring unchanged.

Cost model. Routing replaces $\mathcal{O}(L)$ work with $\sum_h \mathcal{O}(B^{(h)} K_h)$ for selection plus scoring over a small candidate set (§3.6). In per-leaf fusion, only surviving label-wise evaluations are paid; in the global-cosine default, final scoring collapses to a BLAS-grade matrix–vector product over $|\mathcal{C}_q|$ rows of Z , i.e., $\mathcal{O}(|\mathcal{C}_q| d)$ per query. In short, by trading modest per-label memory for locality, sparsity, and parallelism, the pipeline delivers predictable latency and footprint that let XMR scale where flat approaches do not.

3.8 Hybrid X-Linker: Scope of modifications

The refactor preserves the public Application Programming Interface (API) and evaluation contracts, it was only replaced all PECOS-backed units with XMR4EL, so prior experiments re-run unchanged (same data formats, routing semantics, and metrics), ensuring comparability with PECOS-backed results (Figure 2.2 represents the native configuration of the inference without XMR4EL, Figure 3.6 represents the same step but with the thesis model) .

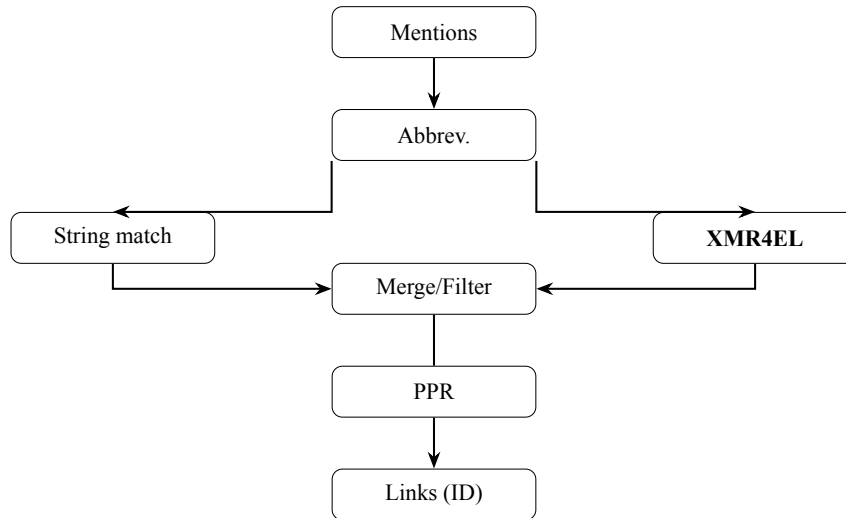


Figure 3.6: Hybrid X-Linker (inference): Mentions $\rightarrow$ Abbrev. $\rightarrow$ {String match || **XMR4EL**} $\rightarrow$ Merge/Filter $\rightarrow$ PPR $\rightarrow$ KB IDs.

Beyond this, we only made engineering updates: pinning and modernising dependencies to run on latest Python, ensuring compatibility with updated third-party packages.

3.9 Results and Discussion

Scope and setup: *XMR4EL* is evaluated purely as a *candidate generator* on **BC5CDR-Disease**. The generator returns up to Top-100 labels per mention; no global top-1/5 assignment and no context-aware reranking are applied. This thesis reports **Hit@ k** (an upper bound on downstream recall) and **DualID@ k** (fraction of mentions for which *all* gold IDs appear within Top- k), with $k \leq 100$ unless stated. Unless noted, training follows automatically labelled **Disease-100/Disease-500**, the label universe is **6k** concepts (memory-bounded), and the test set is $N=1890$ mentions (single-ID: 1,855; two-ID: 33; >2-ID: 2), all present in the KB. Two *selection modes* are used: `per_leaf` (take top- k within each visited leaf, then merge) and `global` (pool all leaf candidates and re-rank once).¹

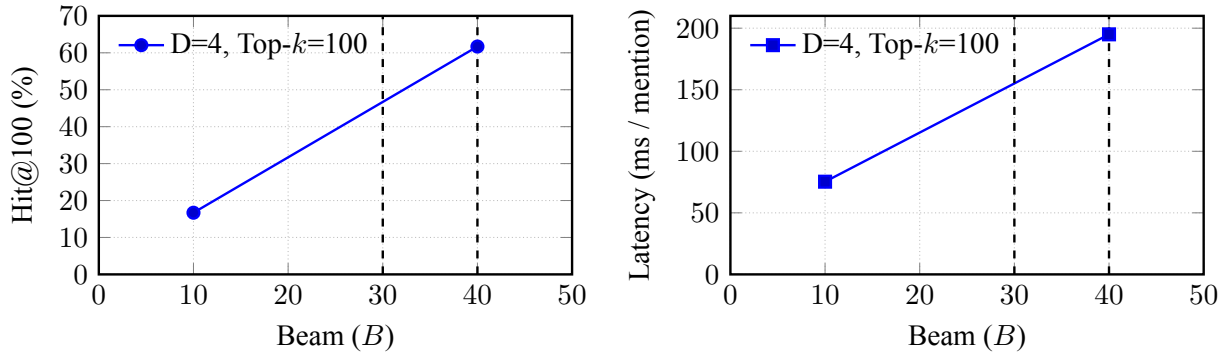
Depth vs. latency: At a common operating point ($B=10$, Top- $k=100$), deeper hierarchies shrink average leaf size and improve recall with modest runtime changes: Hit@100 increases from 9.7% (2 layers) to 16.7% (4 layers), while latency remains practical (144.5 ms $\rightarrow$ 75.3 ms/inst.). Pushing to 5 layers yields a slight regression to 15.2% Hit@100 (67.4 ms/inst.), suggesting an empirical sweet spot near $D=4$ (Table 3.4). Notably, deeper models can be faster than shallower ones at a fixed beam because smaller leaves reduce per-leaf scoring work: in Table 3.4, the 5-layer configuration is faster than the 2-layer and 4-layer configurations (67.4 ms/inst. vs. 144.5 ms/inst. and 75.3 ms/inst.), while the 3-layer configuration is the most efficient overall (52.4 ms/inst.). *Note:* total wall time includes a single-pass encoder and shows benign batching/cache variation across depths.

Layers	#Hit@100	Hit@100 (%)	Time (s)	Latency (ms/inst.)
2	183	9.7	273.13	144.5
3	230	12.2	99.06	52.4
4	316	16.7	142.38	75.3
5	287	15.2	127.40	67.4

Table 3.4: Depth vs. latency at $B=10$ and Top- $k=100$ on BC5CDR-Disease under `per_leaf` selection ($N=1890$); wall-time includes one-pass encoding (batching/cache variation is benign).

Beam quality: Increasing the beam reduces early pruning errors by exploring additional branches, with diminishing returns once the correct branch consistently enters the beam and leaf scoring becomes rate-limiting. Under `per_leaf`, each visited leaf contributes its own top- k , yielding coverage-leaning merged lists (often higher Hit@ k at the same k) but with the risk that populous leaves dominate the budget. Under `global`, a single k is applied after pooling all leaf outputs; this usually favours precision under tight budgets but may discard long-tail golds with modest leaf scores. A beam “knee” is observed for

¹Selection mode affects both latency and Hit@ k : `per_leaf` often boosts coverage at fixed k by aggregating distinct neighbourhoods; `global` applies a single budget after pooling, favouring precision under tight k . Unless otherwise stated, `per_leaf` is used; Hybrid X-Linker experiments use `global`.



(a) Hit@100 as a function of beam width at $D=4$, Top- $k=100$ on BC5CDR-Disease; the dashed band marks the knee region $B \approx 30-40$.

(b) Per-mention latency as a function of beam width at $D=4$, Top- $k=100$ on BC5CDR-Disease; latency grows smoothly through the knee region $B \approx 30-40$.

Figure 3.7: Effect of beam width on Hit@100 and per-mention latency for BC5CDR-Disease at depth $D=4$ and Top- $k=100$, showing an empirical knee near $B \approx 30-40$ (cf. Tables 3.4 and 3.5).

$D=4$ around $B \approx 30-40$, beyond which quality gains flatten while latency grows (Figure 3.7; see also the latency breakdowns in Tables 3.7–3.8).

Coverage vs. list length: At fixed depth and beam ($D=4$, $B=40$), enlarging k markedly improves recall: Hit@ k increases from 6.2% at $k=10$ to 61.7% at $k=100$, while DualID@ k rises from 0.1% to 0.6% (Table 3.5). Beyond $k \approx 100$, Hit@ k flattens whereas DualID@ k continues to increase as all IDs of multi-ID concepts are eventually covered; by $k \in \{200, 500\}$, Hit@ k reaches 100% on this memory-bounded universe. (Hit@1 is 0.0% here because there is no context-aware same-leaf top-1 scorer.)

Top- k	#mentions w/0 golds	#w/1 gold	#w/ ≥ 2 golds	Hit@ k (%)	DualID@ k (%)
1	1890	0	0	0.0	0.0
5	1820	69	1	3.7	0.1
10	1772	117	1	6.2	0.1
50	1262	627	1	33.2	0.1
100	723	1156	11	61.7	0.6
200	0	1869	21	100.0	1.1
500	0	1869	21	100.0	1.1

Table 3.5: Coverage vs. list length for the XMR4EL candidate generator on BC5CDR-Disease at $D=4$, $B=40$ (time = 368.63 s; ≈ 195 ms/mention; $N=1890$); Hit@ k flattens beyond $k \approx 100$ while DualID@ k continues to rise (cf. Figure 3.7).

Impact of training data: Two 4-layer models that differ only in per-label supervision are compared: **Inst-500** (more positives/label) vs. **Inst-100** (fewer). This diagnostic sweep uses a *smaller* test split to accelerate experiments: $N=1285$ (single-ID: 1260; two-ID: 23; >2-ID: 2). Time is measured at $k=1$ and reused across k for each (beam, model). Table 3.6 shows that more instances per label steadily lift *Hit@100* at moderate beams; gains at larger k mainly reflect improved candidate recall rather than top-1 calibration, and Dual-ID improves with k /beam as alias coverage increases.

Model	Beam	Hit@10 (%)	Hit@100 (%)	Gain (10→100)
Inst-500	10	2.0	22.1	+20.1
Inst-500	20	7.9	47.2	+39.3
Inst-100	10	0.1	13.7	+13.6
Inst-100	20	1.6	33.9	+32.3

Table 3.6: Impact of per-label supervision on Hit@k: **Inst-500** vs. **Inst-100** at fixed beams, reporting gains from $k=10$ to $k=100$ on the diagnostic split ($N=1285$); improvements at larger k primarily reflect candidate-recall rather than top-1 calibration.

Latency breakdown: A 1k-label index is instrumented to attribute time across stages (encoding, routing, ranking, and, for `global`, reranking). For depth 3 (Top- $k=20$, batch $N=288$), encoding dominates end-to-end time ($\approx 87\text{--}97\%$), routing/ranking remain single-digit shares, and the additional `global` reranking step is negligible ($< 0.3\%$); per-mention latency lies in the 24–27 ms range across beams (Table 3.7). For depth 4 under the same 1k setting, encoding remains dominant ($\approx 82\text{--}97\%$), routing/ranking scale with beam yet stay in low-teens shares, and per-mention latency increases smoothly with beam—per_leaf from 31.10 ms to 33.19 ms ($B=1 \rightarrow 10$) and `global` from 27.54 ms to 32.56 ms—consistent with additional node expansions and slightly larger leaf candidate sets (Table 3.8).

Mode	Beam	Time (s)					Share (%)				
		Encoding	Routing	Ranking	Reranking	Total	Enc.	Rout.	Rank.	Rerank.	ms/inst.
per_leaf	1	7.5544	0.1288	0.0946	0.0000	7.7778	97.13	1.66	1.22	0.00	27.01
per_leaf	5	6.4890	0.4775	0.3275	0.0000	7.2940	88.98	6.55	4.49	0.00	25.33
per_leaf	10	6.8778	0.4864	0.5147	0.0000	7.8789	87.31	6.17	6.53	0.00	27.36
global	1	6.6842	0.1237	0.0879	0.0080	6.9039	96.82	1.79	1.27	0.12	23.97
global	5	6.6804	0.4795	0.2913	0.0155	7.4667	89.47	6.42	3.90	0.21	25.93
global	10	6.2720	0.4832	0.4610	0.0203	7.2365	86.67	6.68	6.37	0.28	25.13

Table 3.7: Latency breakdown by stage (encoding, routing, ranking, and reranking for `global`) on a 1k-label index at $D=3$, Top- $k=20$, batch $N=288$; encoding dominates ($\approx 87\text{--}97\%$), and reranking is negligible ($< 0.3\%$).

Mode	Beam	Time (s)					Share (%)				
		Encoding	Routing	Ranking	Reranking	Total	Enc.	Rout.	Rank.	Rerank.	ms/inst.
per_leaf	1	8.6591	0.2180	0.0791	0.0000	8.9561	96.68	2.43	0.88	0.00	31.10
per_leaf	5	8.1680	1.0586	0.2052	0.0000	9.4318	86.60	11.22	2.18	0.00	32.75
per_leaf	10	7.8919	1.3072	0.3596	0.0000	9.5587	82.56	13.68	3.76	0.00	33.19
global	1	7.6341	0.2145	0.0751	0.0093	7.9329	96.23	2.70	0.95	0.12	27.54
global	5	7.4801	1.0389	0.2033	0.0171	8.7395	85.59	11.89	2.33	0.20	30.35
global	10	7.6869	1.3009	0.3622	0.0259	9.3759	81.99	13.87	3.86	0.28	32.56

Table 3.8: Latency breakdown by stage on a 1k-label index at $D=4$, Top- $k=20$, batch $N=288$; encoding remains dominant ($\approx 82\text{--}97\%$), with smooth latency growth as beam increases (cf. Table 3.7).

Hybrid X-Linker (BC5CDR; global selection): Under global selection and used as *bare retrieval heads*, PECOS outperforms XMR4EL (0.7803 vs. 0.1307). Enabling abbreviation detection, string matching, and PPR improves both systems—PECOS to **0.8307** and XMR4EL to 0.6171—while PECOS remains ahead, likely due to stronger base recall and an index structure well aligned with its training objective (Table 3.9).

Module	BC5CDR
PECOS	0.7803
PECOS +abbrev detect + SM + PPR	0.8307
XMR4EL	0.1307
XMR4EL +abbrev detect + SM + PPR	0.6171

Table 3.9: Hybrid X-Linker on BC5CDR-Disease under global selection: swapping PECOS for XMR4EL as the retrieval head and the effect of enabling abbreviation detection, string matching, and PPR.

Beyond absolute scores, XMR4EL serves as a *sandbox* for Hybrid X-Linker: it permits easy swapping of components (e.g., testing a LightGBM classifier vs. a transformer ranker), experimenting with alternative clustering algorithms for label partitioning, or evaluating different text encoders for embedding generation. This modularity enables rapid ablations and controlled comparisons within the same pipeline—capabilities that are comparatively constrained in PECOS.

Limitations of the system: There are several practical limitations. First, scores are not state of the art in the Hybrid X-Linker setting. Second, the current candidate generator does *not* employ a learned reranking model; final ordering relies on cosine similarity, which caps top-1 precision relative to cross-encoder or listwise rerankers. Third, memory usage can be substantial: handling large corpora with sparse or dense matrices requires significant RAM, and models without efficient sparse support consume additional

memory. Fourth, GPU acceleration is limited in scope: XMR4EL runs fully on CPU by default; only the encoders are accelerated when a CUDA environment is available. Fifth, while XMR4EL integrates GPU-accelerated components from `rapids.ai`, some of these models expose fewer tunable hyperparameters than their CPU counterparts, which can constrain fine-grained optimization. These constraints motivate the ablation studies, reranking integration, and system optimizations outlined in the future work.

Headline numbers: On automatically labelled disease corpora (*Inst-100/Inst-500* for training; BC5CDR for testing), a **4-layer** hierarchy reaches **61.7% Hit@100** at **~195 ms/mention** with **beam=40**, exhibiting a speed–quality knee near $B=30\text{--}40$. Increasing per-label positives from 100 to 500 yields **+8–13 points Hit@100** at practical beams, and additional depth improves recall at matched latency by shrinking leaf scopes.

Chapter 4

Conclusion

The thesis examined *XMR4EL* as a hierarchical candidate generator for biomedical entity linking under ontology-scale constraints.

In summary: a four-layer hierarchy with beam 30–40 and Top- $k=100$ provides a practical quality–latency trade-off (e.g., 61.7% Hit@100 at ~ 195 ms/mention), supervision density materially improves recall, and PECOS remains stronger as a bare retrieval head in Hybrid X-Linker while *XMR4EL* offers superior modularity and controllable latency.

The motivation in Chapter 1 established the practical need for high-recall, latency-aware retrieval when normalising biomedical mentions at corpus scale, and introduced a modular pipeline where candidate generation sets the ceiling for downstream linking. Within this framing, Chapter 3 presented the *XMR4EL* approach: a hybrid sparse–dense representation, label geometry via PIFA, and a hierarchical routing-and-ranking scheme with explicit control over depth, beam width, and selection mode. Chapter 2 positioned this design in the context of prior work on candidate generation and extreme multi-label retrieval, highlighting complementary strengths of partition-based retrieval (e.g., PECOS) and hierarchical routing.

The empirical study in Section 3.9 characterised the quality–latency trade-offs across depths and beams on BC5CDR-Disease, using *Hit@k* as an upper bound on recall and *DualID@k* to capture complete coverage for multi-ID concepts. A consistent pattern emerged: increasing depth shrank leaf scope and improved recall at comparable budgets up to an observed sweet spot around four layers, beyond which over-partitioning occasionally reduced candidate coverage. Beam width mitigated early pruning but exhibited diminishing returns, with an empirical knee near $B \approx 30\text{--}40$ where gains flattened while latency grew. Under a representative operating point ($D=4$, $B=40$, Top- $k=100$), the generator reached 61.7% Hit@100 at approximately 195 ms/mention, illustrating that hierarchical routing can provide practical recall under tight latency constraints. Supervision density also mattered: models trained with more instances per label (Inst-500) achieved higher Hit@100 at moderate beams, with improvements at larger k primarily reflecting enhanced candidate recall rather than top-1 calibration; DualID benefits were consistent with broader alias coverage.

The relationship to partition-based retrieval was made concrete in the Hybrid X-Linker setting (Figure 2.2), where global selection budgets are applied across pooled candidates. In this configuration, PECOS remained stronger as a bare retrieval head and after enabling abbreviation detection, string matching, and PPR, indicating a present advantage in base recall that aligns with its indexing and objective. At the same time, *XMR4EL* offered transparent routing, controllable latency envelopes, and modularity that are attractive for systems integration, particularly when candidate budgets must be tuned to application latency targets.

Methodological choices in Chapter 3 and evaluation practices in Section 2.6 were tempered by constraints that bound external validity. The label universe was memory-bounded (6k concepts) to ensure deterministic, reproducible experiments; training relied on automatically labelled instances; candidate generation was intentionally evaluated without document context or a cross-encoder reranker; and the current ordering relies on cosine similarity rather than a learned listwise objective. Section 1.4.1 detailed the steps taken to make these choices auditable and portable. Under these conditions, the reported *Hit@k* values should be interpreted as ceilings for downstream EL accuracy rather than final end-to-end performance. All stated objectives were delivered: a reproducible hierarchical generator was implemented; design axes (depth, beam, selection mode) were systematically explored; supervision-density effects were quantified; latency was instrumented and attributed; and integration into Hybrid X-Linker was demonstrated, even though the scores are not state of the art.

From a practical standpoint, this study suggests a default operating regime that balances coverage and responsiveness: four hierarchy layers, beam in the 30–40 range, and Top- $k=100$ for candidate lists, with `per_leaf` selection for standalone generation and `global` selection when integrated into pipelines that centralise budgets across leaves. Where feasible, increasing per-label supervision toward Inst-500 is recommended to raise candidate recall at moderate beams. These settings align with the latency breakdowns observed in the 1k-label instrumentation, where encoding dominated end-to-end time and reranking overhead under `global` was negligible.

In future work, limitations will be addressed directly. A learned *reranking* stage (e.g., cross-encoder or listwise ranker) will replace cosine-only ordering to improve top-1 precision; *memory* footprint will be reduced via vector quantisation, mixed sparse–dense storage, and pruning of low-utility features; the *memory-bounded label universe* will be relaxed with scalable indexing to recover long-tail coverage; reliance on *automatically labelled data* will be mitigated through cleaner supervision and data curation; *lightweight context* will be injected prior to routing to improve same-leaf placement; limited *GPU acceleration* and *few hyperparameters* in some `rapids.ai` components will be countered by expanded GPU back-ends and tunable alternatives; and targeted *ablations* will isolate contributions of representation blocks, routing depth, beam budgeting, and selection mode. Complementary *error analysis* will focus on entity families with elevated difficulty—especially complex or composite forms (e.g., *ErbB-2/ErbB-3 heterodimer*)—to guide improvements in alias coverage, clustering, and leaf assignment.

References

- Babbar, R. and Schölkopf, B. (2017). Dismec: Distributed sparse machines for extreme multi-label classification. In *Proceedings of the tenth ACM international conference on web search and data mining*, pages 721–729. [11](#)
- Beygelzimer, A., Langford, J., Lifshits, Y., Sorkin, G., and Strehl, A. L. (2014). Conditional probability tree estimation analysis and algorithms. *arXiv preprint arXiv:1408.2031*. [11](#), [15](#)
- Bodenreider, O. (2004). The unified medical language system (umls): integrating biomedical terminology. *Nucleic acids research*, 32(suppl_1):D267–D270. [10](#)
- Chang, W.-C., Yu, H.-F., Zhong, K., Yang, Y., and Dhillon, I. S. (2020). Taming pretrained transformers for extreme multi-label text classification. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3163–3171. [11](#), [14](#), [15](#)
- Davis, A. P., Wieggers, T. C., Johnson, R. J., Sciaky, D., Wieggers, J., and Mattingly, C. J. (2023). Comparative toxicogenomics database (ctd): update 2023. *Nucleic acids research*, 51(D1):D1257–D1262. [5](#), [10](#), [17](#)
- Davis, A. P., Wieggers, T. C., Rosenstein, M. C., and Mattingly, C. J. (2012). Medic: a practical disease vocabulary used at the comparative toxicogenomics database. *Database*, 2012:bar065. [5](#), [10](#), [17](#)
- Doğan, R. I., Leaman, R., and Lu, Z. (2014). Ncbi disease corpus: a resource for disease name recognition and concept normalization. *Journal of biomedical informatics*, 47:1–10. [10](#)
- Garda, S. and Leser, U. (2024). Belhd: improving biomedical entity linking with homonym disambiguation. *Bioinformatics*, 40(8):btae474. [11](#), [17](#)
- Garda, S., Weber-Genzel, L., Martin, R., and Leser, U. (2023). Belb: a biomedical entity linking benchmark. *Bioinformatics*, 39(11):btad698. [11](#)
- Gruber, T. (1993). What is an ontology. [9](#)

- Gu, Y., Tinn, R., Cheng, H., Lucas, M., Usuyama, N., Liu, X., Naumann, T., Gao, J., and Poon, H. (2021). Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare (HEALTH)*, 3(1):1–23. [13](#)
- Islamaj, R., Leaman, R., Kim, S., Kwon, D., Wei, C.-H., Comeau, D. C., Peng, Y., Cissel, D., Coss, C., Fisher, C., et al. (2021). Nlm-chem, a new resource for chemical entity recognition in pubmed full text literature. *Scientific data*, 8(1):91. [10](#)
- Jain, H., Balasubramanian, V., Chunduri, B., and Varma, M. (2019). Slice: Scalable linear extreme classifiers trained on 100 million labels for related searches. In *Proceedings of the twelfth ACM international conference on web search and data mining*, pages 528–536. [11](#), [15](#)
- Jasinska, K., Dembczynski, K., Busa-Fekete, R., Pfannschmidt, K., Klerx, T., and Hullermeier, E. (2016). Extreme f-measure maximization using sparse probability estimates. In *International conference on machine learning*, pages 1435–1444. PMLR. [11](#), [15](#)
- Jiang, T., Wang, D., Sun, L., Yang, H., Zhao, Z., and Zhuang, F. (2021). Lightxml: Transformer with dynamic negative sampling for high-performance extreme multi-label text classification. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 7987–7994. [2](#), [3](#)
- Khandagale, S., Xiao, H., and Babbar, R. (2020). Bonsai: diverse and shallow trees for extreme multi-label classification. *Machine Learning*, 109(11):2099–2119. [2](#), [11](#), [14](#)
- Kolitsas, N., Ganea, O.-E., and Hofmann, T. (2018). End-to-end neural entity linking. In Korhonen, A. and Titov, I., editors, *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 519–529, Brussels, Belgium. Association for Computational Linguistics. [3](#), [16](#)
- Leathart, T., Frank, E., Holmes, G., and Pfahringer, B. (2017). Probability calibration trees. In *Asian conference on machine learning*, pages 145–160. PMLR. [16](#), [17](#)
- Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., and Kang, J. (2020). Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240. [13](#)
- Li, J., Sun, Y., Johnson, R. J., Sciaky, D., Wei, C.-H., Leaman, R., Davis, A. P., Mattingly, C. J., Wieggers, T. C., and Lu, Z. (2016). Biocreative v cdr task corpus: a resource for chemical disease relation extraction. *Database*, 2016. [4](#), [6](#), [10](#), [18](#)
- Lipscomb, C. E. (2000). Medical subject headings (mesh). *Bulletin of the Medical Library Association*, 88(3):265. [10](#)
- Liu, F., Shareghi, E., Meng, Z., Basaldella, M., and Collier, N. (2021). Self-alignment pretraining for biomedical entity representations. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tur, D.,

- Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., and Zhou, Y., editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4228–4238, Online. Association for Computational Linguistics. [13](#)
- Liu, T.-Y. et al. (2009). Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331. [15](#)
- Luo, L., Lai, P.-T., Wei, C.-H., Arighi, C. N., and Lu, Z. (2022). Biored: a rich biomedical relation extraction dataset. *Briefings in Bioinformatics*, 23(5):bbac282. [10](#), [18](#)
- Mohan, S. and Li, D. (2019). Medmentions: A large biomedical corpus annotated with {umls} concepts. In *Automated Knowledge Base Construction (AKBC)*. [2](#), [4](#), [10](#)
- Neumann, M., King, D., Beltagy, I., and Ammar, W. (2019). Scispacy: fast and robust models for biomedical natural language processing. *arXiv preprint arXiv:1902.07669*. [10](#)
- Peng, Y., Yan, S., and Lu, Z. (2019). Transfer learning in biomedical natural language processing: an evaluation of bert and elmo on ten benchmarking datasets. *arXiv preprint arXiv:1906.05474*. [2](#)
- Prabhu, Y., Kag, A., Harsola, S., Agrawal, R., and Varma, M. (2018). Parabel: Partitioned label trees for extreme classification with application to dynamic search advertising. In *Proceedings of the 2018 World Wide Web Conference*, pages 993–1002. [2](#), [11](#), [14](#)
- Ramos, J. et al. (2003). Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, volume 242, pages 29–48. Citeseer. [13](#)
- Robertson, S., Zaragoza, H., et al. (2009). The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389. [13](#)
- Ruas, P., Gallego, F., Veredas, F. J., and Couto, F. M. (2024). Hybrid x-linker: Automated data generation and extreme multi-label ranking for biomedical entity linking. *arXiv preprint arXiv:2407.06292*. [4](#), [6](#), [8](#), [17](#), [18](#)
- Shen, W., Wang, J., and Han, J. (2014). Entity linking with a knowledge base: Issues, techniques, and solutions. *IEEE Transactions on Knowledge and Data Engineering*, 27(2):443–460. [1](#), [2](#), [3](#), [16](#)
- Sung, M., Jeon, H., Lee, J., and Kang, J. (2020). Biomedical entity representations with synonym marginalization. *arXiv preprint arXiv:2005.00239*. [17](#)
- Tagami, Y. (2017). Annexml: Approximate nearest neighbor search for extreme multi-label classification. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 455–464. [11](#), [15](#)

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30. [13](#)
- Wei, C.-H., Allot, A., Lai, P.-T., Leaman, R., Tian, S., Luo, L., Jin, Q., Wang, Z., Chen, Q., and Lu, Z. (2024). Pubtator 3.0: an ai-powered literature resource for unlocking biomedical knowledge. *Nucleic Acids Research*, 52(W1):W540–W546. [5](#)
- Wu, L., Petroni, F., Josifoski, M., Riedel, S., and Zettlemoyer, L. (2020). Scalable zero-shot entity linking with dense entity retrieval. In Webber, B., Cohn, T., He, Y., and Liu, Y., editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6397–6407, Online. Association for Computational Linguistics. [15](#), [16](#)
- Yu, H.-F., Zhong, K., Dhillon, I. S., Wang, W.-C., and Yang, Y. (2019). X-bert: extreme multi-label text classification using bidirectional encoder representations from transformers. [11](#)
- Yu, H.-F., Zhong, K., Zhang, J., Chang, W.-C., and Dhillon, I. S. (2022). Pecos: Prediction for enormous and correlated output spaces. *Journal of Machine Learning Research*, 23(98):1–32. [4](#), [11](#), [12](#), [13](#), [14](#), [15](#), [25](#)
- Zhang, J., Chang, W.-C., Yu, H.-F., and Dhillon, I. (2021). Fast multi-resolution transformer fine-tuning for extreme multi-label text classification. *Advances in Neural Information Processing Systems*, 34:7267–7280. [11](#), [12](#), [13](#), [14](#), [15](#)

Appendix A

Model Configurations and Experimental Setup

Vectorizer (TF-IDF)

- n -grams: [1, 2]
- `max_features` = 30,000
- `max_df` = 0.98
- `use_idf`, `smooth_idf`
- ℓ_2 normalisation

Dimensionality Reduction

- TruncatedSVD with 1,000 components
- Randomized solver; 5 power iterations

Transformer (Mention Encoder)

- BioBERT encoder
- Fixed sparse projection for unified sparse features

Clustering (Hierarchy Construction)

- Balanced spherical k -means
- $k = 6$, cosine distance
- $\text{tol} = 10^{-4}$, $\text{iter_limit} = 1000$
- $\text{cut_half_cluster} = \text{True}$

Matcher

- `SGDClassifier` with log-loss
- ℓ_1 regularisation, early stopping
- $\text{max_iter} = 1000$

Ranker

- `SGDClassifier` with hinge loss
- ℓ_1 regularisation
- $\text{learning_rate} = \text{adaptive}$, $\eta_0 = 0.01$
- Trained with MTFN $\rightarrow$ MMAN
- $\text{ranker_every_layer} = \text{True}$

Negatives (Curriculum)

- Warm-up epochs: $E_{\text{warm}} = 3$
- Warm ratios: (0.70, 0.30)
- Hard ratios: (0.60, 0.30, 0.10)
- $\text{neg_mult} = 15$
- $\text{seed} = 42$